Advanced Algorithms: Notes

by Matthew Cai

1. Big O Notation

> Very short notes for the first chapter; not much to talk about.

Big O Notation seeks to estimate the runtime of the algorithm in terms of the input size. The main things to remember are...

- Constants don't matter
- If we have a series of multiple terms such as $n + n^2 + n^3$, only n^3 matters, i.e. the most expensive time complexity
  Formally, let $f(n)$ and $g(n)$ be functions from $\mathbb{Z}^+ \to \mathbb{R}$. Then, $f = O(g) \iff \exists c \in \mathbb{Z}^+$ that is constant such that $\forall n \in \mathbb{Z}^+$ we have $f(n) \leq cg(n)$.

2. Divide and Conquer

> **Divide and Conquer...**
>
> - Breaks a problem into *subproblems* which are the same problem but smaller
> - *Recursing* through the subproblems and subsequently combining their answers

2.1 Multiplication

The product of two complex numbers can be reduced to *3* multiplications instead of the usual 4. (Gauss)

$$(a+bi)(c+di) = ac - bd + (bc+ad)i = (a+b)(c+d) - ac - bd$$

This small constant factor improvement seems negligible; however, when applied at *every step* in a recursion, it has a noticeable impact on time complexity.

We can apply the same concept to integer multiplication. Let x, y be two n-bit integers, and, for convenience, let $n = 2^k,\ k \in \mathbb{N}$. We now split x, y into two chunks of equal bit length:

$$x = 2^{n/2}x_L + x_R \qquad y = 2^{n/2}y_L + y_R$$

And, following Gauss's idea, expand and simplify the multiplication:

$$\begin{aligned} xy &= 2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R && (1) \\ &= 2^n x_L y_L + 2^{n/2}[(x_L + x_R)(y_L + y_R) - x_L y_L - x_R y_R] + x_R y_R && (2) \end{aligned}$$

Note that we only have to perform *3* multiplications:

$$x_L y_L \qquad x_R y_R \qquad (x_L + x_R)(y_L + y_R)$$

Let $T(n)$ represent the overall running time of the multiplication algorithm, provided an n-bit input. Expression (1) then has a **recurrence relation** of

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

since each multiplication $(x_L y_L, x_R y_L, x_L y_R, x_R y_R)$ should take $T\left(\frac{n}{2}\right)$, since its an $\frac{n}{2}$-bit multiplication, and both summing n-bit numbers and left-shifting (the 2^k coefficients) are linear operations in n.

Deriving the explicit form of this recurrence relation results in $T(n) = O(n^2)$, which provides no improvement over the standard multiplication method taught in school.

In contrast, expression (2) has the recurrence relation of

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

since it has 3 multiplications, and the rest of the operations are still linear in n, as previously stated.

Deriving the explicit form of this recurrence relation results in $T(n) = O(n^{1.59})$, a significant improvement! But why?

Consider the tree formed by the recursion. At each successive level, the number of subproblems *triple* (3 multiplications), but the size of each subproblem is *halved* ($\frac{n}{2}$ bit length). That is,

- The tree's height is $\log_2 n$
- The **branching factor** is 3

At depth k in the tree, there are 3^k subproblems of size $n/2^k$. For each subproblem, a linear amount of work is necessary to identify the next subproblems and combine the answers. In other words, the time spent at depth k (and *only* depth k) is

$$3^k \times O\left(\frac{n}{2^k}\right) = \left(\frac{3}{2}\right)^k \times O(n)$$

In other words, the overall time complexity can be represented by a finite geometric series.

$$T(n) = O(n) + \left(\frac{3}{2}\right)O(n) + \cdots + \left(\frac{3}{2}\right)^{\log_2 n} O(n) \qquad (3)$$

Since the common factor of the geometric series is greater than 1, the runtime is **dominated** by the last term of the series, or the last level of subproblems. This is because this series increases extremely quickly from term to term. Thus,

$$T(n) = \left(\frac{3}{2}\right)^{\log_2 n} O(n) = O(3^{\log_2 n}) \times O\left(\frac{n}{2^{\log_2 n}}\right) = O(3^{\log_2 n})$$

This can actually be rewritten as $O(n^{\log_2 3})$ by applying chain rule (for logarithms, not derivatives) in reverse:

$$O(3^{\log_2 n}) = O(3^{(\log_2 3)(\log_3 n)}) = O(n^{\log_2 3}) \approx \boxed{O(n^{1.59})}$$

**Note that, without Gauss's trick, the derivation comes out to $4^{\log_2 n} = n^2$, i.e. no improvement*

2.2 Recurrence Relations

Divide and conquer algorithms all follow the same pattern. They split a problem of size n into a subproblems of size $\frac{n}{b}$, and then combine these answers in $O(n^d)$ time. This allows us to write their runtimes recursively with the equation

$$T(n) = aT(\lceil n/b \rceil) + O(n^d)$$

We can summarize this in a general "Master Theorem" that *explicitly* describes the time complexity of a divide and conquer algorithm parameterized by n, a, b, d.

> **Master Theorem**
> If $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ for constants $a > 0, b > 1, d \geq 0$, then
>
> $$T(n) = \begin{cases} O(n^d), & d > \log_b a \\ O(n^d \log n), & d = \log_b a \\ O(n^{\log_b a}), & d < \log_b a \end{cases}$$

Proof. We follow the previous derivation for expression (3) to find the finite geometric series

$$T(n) = O(n^d) + \left(\frac{a}{b^d}\right) O(n^d) + \cdots + \left(\frac{a}{b^d}\right)^{\log_b n} O(n^d)$$

**Ensure this makes sense before proceeding. The finer details of derivation are purposely left out to force readers to think here :)*

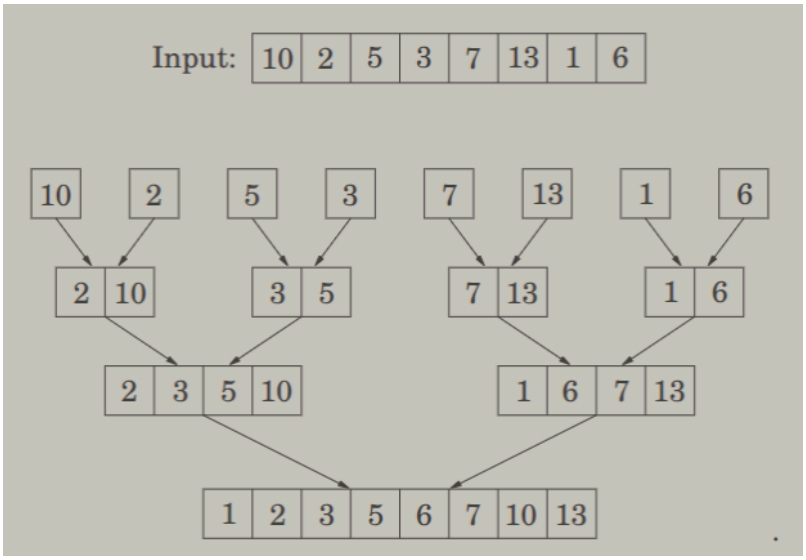
Case 1: $\frac{a}{b^d} < 1$
The series is decreasing. Since it is geometric, the first term dominates the sum, i.e. the time complexity is $O(n^d)$.

Case 2: $\frac{a}{b^d} = 1$
The terms of the series are all just $O(n^d)$. There are $O(\log n)$ terms in the series, so the time complexity is $O(n^d \log n)$.

Case 3: $\frac{a}{b^d} > 1$
The series is increasing. This is the same as the multiplication algorithm case. Thus, the last term dominates the sum, i.e. the time complexity is $O(n^{\log_b a})$, as derived previously.

2.3 Mergesort

Mergesort is one of the most widely used sorting algorithms. The algorithm is a simple application of divide-and-conquer. It starts by splitting the array into halves, and recursing mergesort on each half. This propagates all the way down to **singletons**, i.e. arrays of size one. Then, we traverse back up the recursion step, merging the two arrays returned from our recursive calls into one sorted array. A diagram is displayed below.



Merging can be done in $O(n)$, the subtree splits in 2 at every level, and the subproblem size halves between levels. Thus, the recurrence relation is

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Which comes out to be $O(n \log n)$.

Mergesort can actually be made iterative by queuing all singleton arrays at the beginning, and at each step removing popping two arrays, merging them, and pushing them into the queue.

Once can actually note that the time complexity of general sorting (not including specialized sorting algorithms like radix sort) is $\Omega(n \log n)$, i.e. lower bound. This is derived by considering the number of comparisons required for sorting an array by creating a binary tree with all possible permutations as the leaves and comparisons as the intermediary (non-root and non-leaf) nodes. Thus, the depth of the tree (number of comparisons required) is at least $\log(n!)$. By **Stirling's Formula**,

$$n! \approx \sqrt{\pi\left(2n + \frac{1}{3}\right)} \cdot n^n \cdot e^{-n}$$

Thus, $\log(n!) \geq n \log n$, and sorting's time complexity is at least $n \log n$. Thus, mergesort is optimal!

2.4 Medians

The naive solution to find a median is sorting. However, this has $O(n \log n)$ time complexity. We can do better!

Randomized

Description

We can use a *randomized* divide-and-conquer algorithm. It is known as **quickselect**, and can actually be used for finding the kth smallest number, not just the median. It is very similar to quicksort.

$\text{QuickSelect}(S : \text{array},\ k : \text{int})$

1. Randomly pick a pivot point v from the array S
2. Split S into three arrays, S_ℓ, S_v, S_r defined as follows:
$$\begin{aligned} S_\ell &= \{a \in S \mid a < v\} \\ S_v &= \{a \in S \mid a = v\} \\ S_r &= \{a \in S \mid a > v\} \end{aligned}$$
3. Return one of the following, which may require recursion:
$$\begin{aligned} k \leq |S_\ell| &\implies \text{QuickSelect}(S_\ell, k) \\ |S_\ell| < k \leq |S_\ell| + |S_v| &\implies \text{Return } v \\ |S_\ell| + |S_v| < k &\implies \text{QuickSelect}(S_r, k - |S_\ell|) - |S_v| \end{aligned}$$
The three arrays S_ℓ, S_v, S_r can be computed from S in $O(n)$ time complexity. And, at each step, the subproblem shrinks to a size of $\max(|\text{S}_\ell|, |\text{S}_\text{r}|)$.

Asymptotic Analysis

Since this is a randomized algorithm, its time complexity is dependent on the choice of pivot.

Worst Case

The worst case scenario occurs when our chosen pivot is always the largest or smallest element of the array. This will cause the subproblem to shrink by only one element each time, which results in a time complexity of

$$n + (n-1) + \cdots + \frac{n}{2} = \Theta(n^2)$$

However, this is probabilistically impossible.

Best Case

The best case scenario occurs when our chosen pivot is always the median. This results in the recurrence relation

$$T(n) = T\left(\frac{n}{2}\right) + O(n)$$

By the Master Theorem, this is just $O(n)$. Again, though, this is extremely unlikely to occur.

Average

Let v be the chosen pivot. Let v be *good* if it is within the interquartile range (25th to 75th) percentile of the array. Good choices of v guarantee that $|S_\ell|, |S_r| \leq \frac{3}{4}|S|$. And we expect to pick, on average, 2 choices of v before getting a good v.

> **Proof**
> Let E be the expected number of choices before a good v. If the current choice is good, we're done. If the current choice is bad, we need to repeat. Thus, $E = 1 + \frac{1}{2}E$. That is, $E = 2$.

Thus, on average, after two split operations, the subproblem will be at most $\frac{3}{4}$ of the original problem size. Therefore, we can write the *average* recurrence relation

$$T(n) \leq T\left(\frac{3n}{4}\right) + O(n)$$

Note that it would be $2O(n)$ since its two split operations, but we ignore constant factors as usual.

Using the Master Theorem, we can conclude that $T(n) = O(n)$. In other words, the algorithm should return the correct answer in **linear** time complexity, on average.

Deterministic

Description

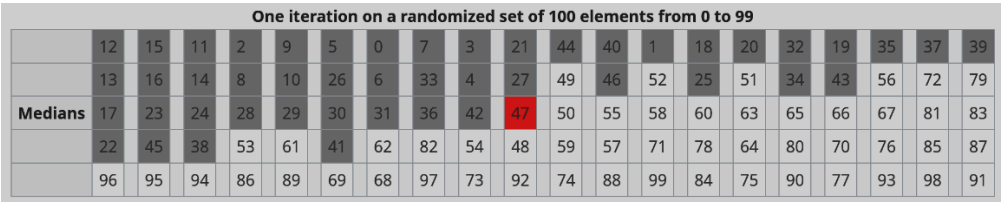
There also exists a *deterministic* divide-and-conquer version of QuickSelect that uses as pivot algorithm known as **median of medians**. It is parameterized by x, the size of each subarray constructed. x must be at least 4.

$\text{MedianOfMedians}(S : \text{array},\ x : \text{int})$

1. Split the array into contiguous subarrays of size k.
2. Find the median of each array by calling the algorithm on each array. (Or, alternatively, using a naive subroutine since x is generally small).
3. Find the median of the medians m by recursively calling the algorithm on the array of medians.
4. The median of medians is used to choose to recurse on at most 70% of the array, with the exact proportion dependent on the value of k in QuickSelect (assuming $x = 5$). See below for an explanation of how the split works.

Asymptotic Analysis

We will choose $x = 5$ for convenience of analysis, and analyze the time complexity of QuickSelect when using MedianOfMedians as a pivot selection algorithm.



First, we will consider how we are able to eliminate 30% of the array. Consider the image above. Here, an array of 100 elements has been divided into 20 subarrays of size 5. The medians are all in row 3, and the median of medians is the element highlighted in red. Everything grayed out is less than the median of medians.

The properties of the median of medians *guarantees* that everything in the top left corner bounded by the row and column of the median of medians (row 1 to 3, columns 1 to 10). This makes sense because the median of medians is greater than all the median of all subarrays of size 5 before it, and the median of each subarray is greater than the elements above it in the table.

In other words, finding the median of medians allows us to identify with certainty the lower 30% of array elements. A similar thought process follows to identify the upper 30% of array elements.

Therefore, instead of fully partitioning the array based on a pivot, we can use the median of medians as a pivot and subsequently recurse on either the lower or upper of 70% of the array based on k. We cannot recurse on the lower/upper 30% of the array because there exist some values not within those regions that are less/greater than the median of medians, respectively.

Thus, the recurrence relation is

$$T(n) \leq T(n/5) + T\left(\frac{7}{10}n\right) + O(n)$$

The $T(n/5)$ is for finding the median of the medians, the $T\left(\frac{7}{10}n\right)$ is for the next subproblem, and the $O(n)$ is for the partitioning.

To derive the explicit time complexity it helps to draw out the tree for a recurrence relation of the form $T(n) = T(an) + T(bn)$. I'm not dedicated enough to draw a tree, but you'll notice that the total sum of the subproblem sizes at the kth level is $(a+b)^k n$. Since the combination step (partitioning) is $O(n)$ complexity, the time complexity of the kth level is $O((a+b)^k n)$. Note that this means the time complexity is simply a geometric series.

$$T(n) = O(n) + O((a+b)n) + \cdots + O((a+b)^k n)$$

Where k here represents the last level. Recall that, in previous proofs, either the first or last term dominates in a finite geometric series. Here, the common ratio depends on $a+b$. For our recurrence relation, $a+b=\frac{1}{5}+\frac{7}{10}=\frac{9}{10}<1$. Therefore, the first term dominates. That is,

$$T(n)=\Theta(n)$$

2.5 Matrix Multiplication

We will assume we are multiplying 2 $n\times n$ matrices. The naive algorithm for matrix multiplication is $O(n^3)$, as there are $O(n^2)$ entries to be computed, and each requires $O(n)$ time.

We can form a naive divide-and-conquer algorithm too, by splitting each matrix into $n/2\times n/2$ blocks, and computing their products before combining. Unfortunately, the recurrence relation is

$$T(n)=8T(n/2)+O(n^2)$$

Which comes out to $O(n^3)$. However, Volker Strassen came up with a very clever improvement on this divide-and-conquer algorithm. In particular, he figured out how to compute the product of two $n\times n$ matrices X and Y from just *seven* $n/2\times n/2$ subproblems.

$$XY=\begin{bmatrix}P_5+P_4-P_2+P_6 & P_1+P_2\\ P_3+P_4 & P_1+P_5-P_3-P_7\end{bmatrix}$$

Where

$$\begin{aligned}P_1&=A(F-H) & P_5&=(A+D)(E+H)\\ P_2&=(A+B)H & P_6&=(B-D)(G+H)\\ P_3&=(C+D)E & P_7&=(A-C)(E+F)\\ P_4&=D(G-E)\end{aligned}$$

The new recurrence relation is

$$T(n)=7T(n/2)+O(n^2)$$

Applying the Master Theorem gives us $O(n^{\log_2 7})\approx O(n^{2.81})$.

Aside: Binary Exponentiation

Naive

The naive exponentiation algorithm to calculate a^n is to just multiply a with itself times. This has a time complexity of $\Theta(n)$, if you consider multiplication as a constant time operation. However, multiplication is not constant time for binary exponentiation! Consider that the time complexity of multiplication grows with respect to the number of digits of the number (the

maximum between the two being multiplied). Let a have m digits. Then a^2 has about $2m$ digits, a^3 has about $3m$ digits, ..., and a^n has about nm digits. Thus, the number of digits grows with respect to n. Therefore, this has time complexity $O(n^{2.59})$.

Binary

Instead of doing that, we can use the following algorithm.

```
def exp(a, n):
    if n % 2 == 1: return a * exp(a*a, n>>2)
    else: return exp(a*a, n>>2)
```

This has $O(\log n)$ time complexity without considering the time complexity of multiplication. Considering the complexity of multiplication, it has time complexity $O(\log n \cdot n^{1.59})$.

Aside: Fibonacci Sequence

Naive

The naive algorithm for the Fibonacci sequence is simply to do a simple recursion.

```
def fib(n):
    if n == 0: return n
    if n == 1: return n
    return fib(n - 1) + fib(n - 2)
```

However, this has exponential time complexity. We proceed via induction.

The recurrence relation is $T(n) = T(n-1) + T(n-2) + O(1)$. We seek to prove that the time complexity is $O(2^n)$.

Base Case: $n = 1$
Trivial.

Inductive Hypothesis: $1 \leq n \leq k$
Assume the time complexity is $O(2^n)$ for $1 \leq n \leq k$.

Inductive Step: $n = k + 1$

$$T(k+1) = T(k) + T(k-1) + O(1) = 2^k + 2^{k-1} + O(1) = O(2^{k+1})$$

Thus, the time complexity is $O(2^n)$. In fact, we can product an even tighter bound of $O(\phi^n)$, where ϕ is the golden ratio. This can be intuitively explained as follows.

Assume $T(n) = a^n$, where $a \in \mathbb{R}^+$. Then,

$$\begin{aligned} a^n &= a^{n-1} + a^{n-2} \\ a^2 &= a + 1 \\ 0 &= a^2 - a - 1 \\ a &= \frac{1 \pm \sqrt{5}}{2} = \phi \end{aligned}$$

Dynamic Programming

We can definitely compute it faster though! Notice how we are recomputing values we already computed? (In other words, calling `Fib` with the same argument multiple times). We can instead **memoize** this, remembering what the output of `Fib` was for that input.

```
def fib(n):
    dp = [0 for _ in range(n + 1)]
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

This allows for linear time complexity... right?

Sorta. If you consider addition to be an $O(1)$ operation, then, yes, this is $O(n)$. Technically, addition is an $O(n)$ operation for the Fibonacci sequence! Let's set some bounds for the Fibonacci sequence.

$$F_n = F_{n-1} + F_{n-2} = 2F_{n-2} + F_{n-3} > 2F_{n-2}$$

But also, since $F_{n-2} < F_{n-1}$,

$$F_n < 2F_{n-1}$$

Therefore, $2^{n/2} < F_n < 2^n$. With this, n is a good approximation of the number of digits of F_n. (Technically, $n/3$, but constant factors don't matter). Therefore, the addition operation actually takes $O(n)$ time complexity for calculating F_n.

We thus must consider the number of steps for the addition operation at every step of the for loop. The total sum is

$$1+2+3+\cdots+n=\frac{n(n+1)}{2}=O(n^2)$$

Thus, the dynamic programming method actually has a time complexity of $O(n^2)$.

Binary Matrix Exponentiation

We can note the following formula for calculating the Fibonacci sequence:

$$\begin{bmatrix}F_{n-1}\\F_n\end{bmatrix}=\begin{bmatrix}0&1\\1&1\end{bmatrix}\begin{bmatrix}F_{n-2}\\F_{n-1}\end{bmatrix}$$

Extrapolating, we get

$$\begin{bmatrix}F_{n-1}\\F_n\end{bmatrix}\begin{bmatrix}0&1\\1&1\end{bmatrix}^n\begin{bmatrix}F_{n-2}\\F_{n-1}\end{bmatrix}$$

Through matrix diagonalization, we can make the time complexity of binary matrix exponentiation essentially equivalent in time complexity to binary integer exponentiation. Thus, the fastest algorithm is actually $O(\log n \cdot n^{1.59})$.

**Note that taking the limit of this matrix exponentiation produces the golden ratio ϕ, as confirmed in the previous algebraic derivation of ϕ.*

Aside: Closest Pair

> Given a set P of n points in a plane, calculate the Euclidean distance between the closest pair of points.

Naive

There is clearly an $O(n^2)$ algorithm that checks the distance between every pair of points, and finds the minimum.

Divide and Conquer

Algorithm

Choose a pivot point (x, y) in the middle of the points in terms of x-coordinate. Divide the graph into two sets L and R, corresponding to points left and right of the pivot, respectively. Then, calculate the closest pair distance in L and the closest pair distance in R by recursing the algorithm on these subproblems. Let d_L, d_R denote these distances. Let $d = \min(d_L, d_R)$. We can prove some properties for this problem.

> **Property 1:** If $p \in L$, $q \in R$, and $\Delta(p,\ q)$ (distance between p and q) is less than d, then $|p.x - \text{pivot}_x| < d$ and $|q.x - \text{pivot}_x| < d$.

Proof: We proceed via proof by contraposition. Let p lie outside the region of the plane where $\text{pivot}_x - d < x < \text{pivot}_x + d$. Since $q \in R$, $\Delta(p.x,\ q.x) > \Delta(p,\ (\text{pivot}_x,\ p.y))$ Then, it is guaranteed that $|p.x - \text{pivot}_x| \geq d \implies \Delta(p,\ q) \geq d$.

□

> **Property 2:** Any tile T that is $\frac{d}{2} \times \frac{d}{2}$ in size that is entirely contained in L or R can have at most one point.

Proof: We proceed via contradiction. Assume $\exists p, q \in S$ and p, q are contained in T. Then, $\Delta(p, q) \leq \sqrt{\left(\frac{d}{2}\right)^2 + \left(\frac{d}{2}\right)^2} = \frac{d}{\sqrt{2}} < d$. That is, this pair of points would have formed the closest pair in L or R, which is a contradiction.

□

> **Property 3:** Let p lie inside the region of the plane where $\text{pivot}_x - d < x < \text{pivot}_x + d$. Then, there are at most 7 points in P within the same region such that $p.y \leq q.y$ and $\Delta(p,\ q)$ *could* be less than d.

Proof: Consider the $2d \times d$ region within the aforementioned $\text{pivot}_x - d < x < \text{pivot}_x + d$ region where p lies in the vertical bottom of the $2d \times d$ region. Subdivide the $2d \times d$ region into 8 $\frac{d}{2} \times \frac{d}{2}$ squares, and arbitrarily assign p to one of the squares it borders. Consider that any point q above this $2d \times d$ region certainly has $\Delta(p,\ q) \geq \Delta(p.y,\ q.y) \geq d$. Therefore, q must exist within one of the other 7 squares $\implies$ there are at most 7 points in P within the same region such that the statement is true.

□

We can now describe the algorithm.

1. First, we preprocess the set of points P, producing a list P_x, the set of points sorted by x, and P_y, the analogue for y.
2. We define a function $\text{Closest}(P_x, P_y)$
    1. Let $n = |P_x|$. If $n \leq 3$, just brute force.
    2. Choose the middle pivot point, "middle" in terms of x. Partition P_x into L_x and R_x in $O(1)$ time. Partition P_y into L_y and R_y in $O(n)$ time. These lists should remain sorted according to their subscript.

3. Recurse on the subproblems. That is, $d_L = \text{Closest}(L_x, L_y)$ and $d_R = \text{Closest}(R_x, R_y)$. Then, $d = \min(d_L, d_R)$,
4. Define the middle $2d \times d$ region (strip) and conquer. We write the pseudocode here:

```
# construct strip, sorted by y
strip = []
for p in Py:
   if abs(p.x - pivot.x) < d:
       strip.append(p)

# process closest pair candidates
k = len(strip)
for i in range(k):
    for j in range(i + 1, k): # this will run at most 7 times
        if strip[j][1] - strip[i][1] >= d:
            break
        d = min(d, distance(strip[i], strip[j]))
```

Time Complexity Analysis

Preprocessing takes $O(n \log n)$ time. Partitioning takes $O(n)$ time. Recursing on the subproblems takes $2T(n/2)$ time. Processing the strip takes $O(n)$ time, with a small constant factor. The recurrence relation is thus

$$T(n) = 2T(n/2) + O(n)$$

By the Master Theorem, $T(n) = O(n \log n)$. Since preprocessing also has $O(n \log n)$ time complexity, the overall algorithm has $O(n \log n)$ time complexity!

□

3. Decompositions of Graphs

3.1 Graph Representations

Adjacency matrix vs. adjacency list yap... not too relevant for an Advanced Algorithms class, in my opinion.

3.2 (Undirected) Depth-First Search

Pseudocode:

```
dfs(G, v):
    visited(v) = true
    previsit(v)
    for edge (v, u) in E:
        if !visited(u): dfs(u)
    postvisit(v)
```

- The vertices and edges traversed by DFS form a tree.
- A disconnected graph will have multiple DFS trees $\rightarrow$ **forest**

Asymptotic Analysis

Two steps:

1. Some fixed amount of work--marking the vertex as visited and then the time complexity of pre/postvisit work.
2. Recurse on adjacent edges if not visited.
   The total work done in step 1 across all vertices is $O(|V|)$ (assuming pre/postvisit work is $O(1)$). The total work done in step 2 is $O(|E|)$, as the algorithm will traverse all edges for each vertex. Thus, the time complexity is $O(|V| + |E|)$; essentially $O(|E|)$.

Connected Components

Just define `previsit` to be

```
previsit(v):
    ccnum[v] = cc
```

Where cc is the identifying index of the CC passed around by DFS.

Pre-order and Post-order

```
previsit(v):
    pre[v] = clock
    clock++

postvisit(v):
    post[v] = clock
    clock++
```

> Note that, for any two nodes u, v, for the intervals $[pre[u], post[u]]$ and $[pre[v], post[v]]$, either
>
> - they are entirely disjoint
> - one is entirely within the other

3.3 (Directed) DFS

- Tree edges are part of the DFS forest
- Forward edges lead from a node to a descendant that is not a direct child (i.e. more than one level apart)
- Back edges lead to an ancestor in the tree
- Cross edges lead to a vertex that is neither a descendant nor an ancestor
  We can characterize these edges by the overlap of their pre/post intervals:

$$\begin{array}{cccc} [_u & [_v &]_v &]_u \end{array} \implies \text{Tree/Forward}$$
$$\begin{array}{cccc} [_v & [_u &]_u &]_v \end{array} \implies \text{Back}$$
$$\begin{array}{cccc} [_v &]_v & [_u &]_u \end{array} \implies \text{Cross}$$

  There is a special type of directed graph known as a **directed acyclic graph** (DAG). This just denotes a directed graph without any cycle. It is possible to test for acylicity in linear time, with a single depth-first search.

> A directed graph has a cycle $\iff$ its DFS reveals a back edge.

In other words, if the DFS reaches a vertex it has already visited before, the directed graph has a cycle.

DAGs are highly applicable to many real world situations. Frequently, it's desirable to *linearize* or *topologically sort* a DAG. This means identifying an ordering of the vertices such that each

edge goes from an earlier to a later vertex in the sequence. All DAGs can be linearized! This is because of a very nice property that derives from the fact that DAGs cannot have back edges:

> In a DAG, every edge leads to a vertex with a lower post number.

Therefore, we have an algorithm that runs in linear time that orders the nodes of a DAG (toposort). We can also conclude that the vertices with the smallest post number must be *sinks*. Similarly, the vertices with the highest post number must be *sources*. This gives us one more property.

> Every DAG has at least one source and one sink.

Thus, an alternate approach to linearization is actually iteratively finding a source, appending it to the sequence, and then deleting it.

3.4 Strongly Connected Components

Description

Strongly connected components apply only to directed graphs. For directed graphs, we state that two nodes u, v are *connected* if there exists a path from u to v *and* a path from v to u. This definition allows us to partition a directed graph into disjoint sets that are denoted as **strongly connected components**. In each set, the vertices are pairwise connected.

Now, if we shrink each SCC into a single "meta-node," and draw an edge from each meta-node to another if there is an edge in the same direction between their underlying components, we can create a "meta-graph" that is, in fact, a DAG. That is,

> The meta-graph constructed by the strongly connected components of any directed graph is always a DAG.

Proof:
Let U, V be two SCCs and let $u \in U$ and $v \in V$. If there exists a bidirectional (back) edge between U and V, then there exists a path from u to v and v to u. This is because, since U and V are SCCs, it is possible for u to traverse to the vertex with an edge to V and then traverse to v from there. The analogue can be said for v to u. This would mean $U \cup V$ is an SCC, a contradiction. Therefore, the meta-graph must be a DAG.

□

This gives insight into the greater structure of a directed graph. In particular, at the top level, there exists a simpler DAG of the graph. Underlying this DAG, though, is the actual directed graph with SCCs in each of the DAG's nodes.

Strongly Connected Components Decomposition

We can actually decompose a directed graph into its SCCs in linear time.

> **Property 1:** If DFS is started at node u, it will terminate precisely when all nodes reachable from u have been visited.

This means that, if we start DFS on a node that lies within an SCC that is a *sink* on the meta-graph, it will visit exactly the SCC, and nothing more.

> **Property 2:** The node that receives the highest post number in a DFS must lie in a *source* SCC.

This is a result of a more general property:

> **Property 3:** if C and C' are SCCs, and there exists an edge from a node in C to a node in C', then the highest post number in C is *larger* than the highest post number in C'.

Proof: There are two cases to consider. If DFS visits component C before C', then all of C and C' will be traversed before the procedure ends. By the properties of DFS, the post number of the first visited node in C will be higher than the highest post number in C'. Otherwise, if C' is visited first, DFS will end after visiting all of C' *but* before visiting any of C, in which case the property follows immediately.

□

We can restate Property 3 as "the SCCs can be linearized by arranging them in decreasing order of their highest post numbers."

Now, consider the *reverse* graph G^R, which is the same as the original graph G but with all edges reversed. G^R has the same connected components as G, but the meta-graph has its edges reversed! By Property 2, if we perform DFS on G^R, the node with the highest post number will be a *source* SCC for the meta-graph of G^R, but will be a *sink* SCC for the meta-graph of the original graph G.

Once we have identified a sink SCC of G, we then delete the sink from the graph. Then, by Property 3, the node with the highest post number among the remaining nodes will belong to another sink SCC of G. Therefore, we can iteratively delete the sink from the graph until there are no nodes left. Thus, we have a linear time algorithm to decompose the graph into SCC DAGs.

More formally, we describe **Kosajaru's Algorithm** for decomposing a directed graph into its **condensation graph**, i.e. the DAG of its SCCs. The following is taken from cp-algorithms,

1. Run DFS 1 or more times on G, computing postorder number for each vertex and yielding a list of the vertices sorted by postorder number.
2. Build the transpose graph G^T, and repeatedly run DFS on the vertices in descending postorder. By the properties described above, the node with the highest postorder number is part of a source SCC of G, but a sink SCC of G^T. Therefore, running DFS on the node with highest postorder number will visit *only* the nodes of this sink SCC of G^T. This essentially extends, to the rest of the nodes, as the algorithm visits the SCCs of G^T in *reverse topological order*. This means it visits the SCCs of G in *topological order*.

4. Paths in Graphs

> Note: Many chapters of the book are omitted here, as I felt they were too simple to be discussed in depth in an Advanced Algorithms class.

4.4 Dijkstra's Algorithm

Priority queue with 4 operations:

- *Insert*: add a new element in sorted order
- *Decrease-key*: decrease the key value of a particular element, changing its position in the queue if necessary (can also be simulated by reinserting into the priority queue and keeping track of visited nodes)
- *Delete-min*: pop the element with the smallest key
- *Make-queue*: build a priority queue out of the given elements (faster operation than one-by-one insertion)

DPV Psuedocode:

```
procedure dijkstra(G,l,s):

Input:
- Graph G=(V,E), directed or undirected
- l = set of edges, positive lengths/weights
- s = start vertex

Output:
- dist array, where dist[u] is the distance from s to u

Algorithm:
for u in V:
    dist[u] = INFTY
    prev[u] = nil
dist[s] = 0

q = makequeue(V)
while !empty(q):
    u = deletemin(H)
    for edge (u,v) in E:
        if dist[v] > dist[u] + l[(u,v)]:
```

```
            dist[v] = dist[u] + l[(u,v)]
            prev[v] = u
            decreasekey(q, v)
```

Proving Dijkstra's algorithm can be done via induction, with the following inductive hypothesis:

> At the end of each iteration of the while loop, the following conditions hold:
>
> 1. There is a value d such that $\forall x \in R$, the set of nodes with known distances, $\text{dist}[x] \leq d$.
> 2. $\forall x \in V$, the value $\text{dist}[u]$ is the length of the shortest path from s to u whose intermediate nodes are constrained to be in R. If no such path exists, the value is ∞.

Dijkstra's algorithm has a runtime of $O((|V| + |E|) \log |V|)$ with a binary heap. With a Fibonacci heap, this can be improved to $O(|V| \log |V| + |E|)$. This is optimal for sparse graphs, i.e. when $|E|$ is fairly small.

4.5 Priority Queue Implementations

Array

$O(1)$ for `insert` and `decreasekey`. But, `deletemin` is $O(n)$.

Binary Heap

Recall 61B! $O(\log n)$ for `decreasekey`, `insert`, and `deletemin` due to "bubbling up" for the first two and "sifting down" for the third.

d-ary Heap

A d-ary heap is just a binary heap except each node has d children. This reduces the height of a tree to $\Theta(\log_d n) = \Theta\left(\frac{\log n}{\log d}\right)$. `insert` and `decreasekey` therefore take $O\left(\frac{\log n}{\log d}\right)$. But, `deletemin` take $O(d \log_d n)$, since d children must be checked while sifting down.

4.6 Shortest Path Algorithms with Negative Edges

Bellman-Ford

Dijkstra's algorithm does not work with graphs that have negative edge weights. This is because Dijkstra relies on the assumption that the `dist` values are always $\geq$ the actual distance, and should eventually all converge to the real distance—at which point, they are considered to have be part of the "known" set of nodes.

With negative edges, this is not necessarily true, as the subsequent discovery of a negative edge might mean there exists a shorter path for a node that is already included in the "known" set of nodes.

The **Bellman-Ford** algorithm is an $O(|V| \cdot |E|)$ procedure that finds shortest paths even in the presence of negative edges, by simply updating all the edges. This avoids the problem that Dijkstra runs into, in which it's possible to update the edges in the wrong order (i.e. negative edge later). The pseudocode of its implementation follows below:

```
procedure bellman-ford(G,l,s):
Input: same as Dijkstra's, but graph must be directed (otherwise, a single
negative edge is already a negative cycle!)
Output: same as Dijkstra's

Algorithm:
for u in V:
    dist[u] = INFTY
    prev[u] = nil
dist[s] = 0

for _ in range(|V| - 1):
    for (u,v) in E:
        if dist[u] < INFTY:
            d[v] = min(d[v], d[u] + l[(u,v)])
```

Notably, the algorithm above still does not work if there exists a **negative cycle**! A negative cycle is simply a cycle that allows infinite reduction of the distance. Why doesn't it work? Well, with a negative cycle, the shortest-path problem is ill-posed. There doesn't really exist a shortest path, since, with a negative cycle, the shortest path can have infinitely negative weights.

Note: to check if there exists a negative cycle in the graph, we can also use Bellman-Ford. Simply run the algorithm for one more iteration—if an update to any vertex's distance is observed, there exists a negative cycle, as Bellman-Ford must end within $|V| - 1$ iterations if there is a shortest path. After all, a shortest path can traverse through at most $|V| - 1$ other nodes before reaching a target node.

Proof of the Bellman-Ford Algorithm:

First, note the correctness of the algorithm for vertices that cannot be reached from the start

node, s, as $\text{dist}[v]$ will remain as ∞.

We now aim to show that, after the ith iteration, the Bellman-Ford algorithm correctly finds all shortest paths whose number of edges $\leq i$. Consider an arbitrary vertex v for which there exists a path from s to v. Let the nodes along the shortest path be denoted $p_0 = s, p_1, \ldots, p_k = v$. Before the first iteration, $\text{dist}[v] = 0$ is known. During the 1st iteration, (p_0, p_1) is checked, and the distance to p_1 is correctly calculated. We can state a similar assertion for the next $k - 1$ nodes. Therefore, the shortest path form s to v will certainly be determined after $|V| - 1$ iterations. In fact, this proof shows that it is *guaranteed* for this to be the case, as a shortest path cannot have more than $|V| - 1$ edges.

Shortest Path Faster Algorithm (SPFA)

This is an improvement on the Bellman-Ford algorithm. It takes advantage of the idea that not all attempts at relaxation will actually result in a change. SPFA maintains a queue with only relaxed vertices that could further relax their neighbors. So, whenever a neighbor is relaxed, the algorithm puts it on the queue. This, thus, avoids visiting all n vertices every iteration ($n - 1$ iterations).

The worst case of this algorithm is equal to $O(|V| \cdot |E|)$, like Bellman-Ford. But, in practice, it is more efficient.

Note also that the algorithm can continue forever if there exists a negative cycle; thus, it's necessary to keep track of how many times each vertex has been relaxed and stop the algorithm when a vertex is relaxed for the nth time.

```
##define f first
##define s second
##define PII pair<int, int>
##define MP make_pair

const int INF = int(1e9);
vector<vector<PII>> adj;

bool spfa(int s, vector<int>& d) {
    int n = adj.size();
    d.assign(n, INF);
    vector<int> cnt(n, 0);
    vector<bool> inq(n, false);
    queue<int> q;
```

```
    d[s] = 0;
    q.push(s);
    inq[s] = true;
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        inq[v] = false;

        for (auto edge : adj[v]) {
            int to = edge.f;
            int len = edge.s; // weight

            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                if (!inq[to]) {
                    q.push(to);
                    inq[to] = true;
                    if (cnt[to] > n)
                        return false; // negative cycle
                }
            }
        }
    }
}
```

Source

4.7 Shortest Paths in DAGs

Since, in any path of a DAG, the vertices are guaranteed to appear in *linearized order*, we can simply linearize the DAG first and then visit the vertices in sorted order, relaxing the edges as we go. This algorithm doesn't even require the edges to be positive, either. We can also find the longest paths in a DAG by the same algorithm, just by negating all edge lengths.

```
procedure dag-shortest-paths(G,l,s)
```

```
Input:
- G=(V,E) : DAG
- l = set of edge weights
- s = starting vertex
Output:
- Same as Dijkstra's

Algorithm:
for u in V:
    dist[i] = INFTY
    prev[u] = nil

dist[s] = 0
toposort(G)
for u in V, linearized:
    for e in E:
        relax(e)
```

5. Greedy Algorithms

Lecture: Task Scheduling

Input is n jobs with start and end times, i.e. $[(s_1, t_1), \ldots, (s_n, t_n)]$. Schedule the maximum number of *non-overlapping jobs*, and return this number.

The solution is to greedily select the job that finishes first!

Proof.
Let $(s_1, t_1), \ldots, (s_r, t_r)$ denote the schedule produced by the greedy algorithm. Now, let $(s'_1, t'_1), \ldots, (s'_\ell, t'_\ell)$ denote a known optimal solution.

> Claim 1: $r \leq \ell$.

This follows from the fact that ℓ is optimal.

> Claim 2: Let $H_i = (s_1, t_1), \ldots, (s_i, t_i), (s'_{i+1}, t'_{i+1}), \ldots, (s'_\ell, t'_\ell)$. Then, H_i is optimal $\forall i \in \{0, \ldots, r\}$.

Base Case: $i = 0$.
H_0 is equivalent to the given optimal solution, so this is, by definition, optimal.

Inductive Hypothesis: $i = k$
Assume that H_i is optimal for $i = k$.

Inductive Step: $i = k + 1$
Note that $t_i < s_{i+1} < t_{i+1} \leq t'_{i+1} < s'_{i+2}$. Therefore, replacing (s'_{i+1}, t'_{i+1}) with (s_{i+1}, t_{i+1}) works just fine, and H_{i+1} is optimal since H_i with optimal. Thus, the greedy algorithm's solution is optimal.

□

Runtime Analysis.
This algorithm requires first sorting the tasks by finishing time in ascending order, and then iterating over them in linear time. Therefore, the overall time complexity is $O(n \log n)$.

5.1 Minimum Spanning Trees

Given an undirected graph $G = (V, E)$ with edges weights w_e, the minimum spanning tree (MST) is a tree $T = (V, E')$ with $E' \subseteq E$ that minimizes

$$\text{weight}(T) = \sum_{e \in E'} w_e$$

Aside: Tree Properties

A tree is defined a an undirected graph that is connected an acyclic.

> A tree with $|V| = n$ has $|E| = n - 1$.

Consider the construction of a tree, one edge at a time, starting from n components, one vertex each. Each connection reduces the number of components by 1 (otherwise, it produces a cycle). Thus, $n - 1$ edges leads to 1 component.

> Any connected, undirected graph $G = (V, E)$ with $|E| = |V| - 1$ is a tree. (Converse of previous).

Simply show G is acyclic. While the graph contains a cycle, remove an edge from the cycle. The process terminates with $G' = (V, E'), E' \subseteq E$ that is acyclic and connected. Thus, G is a tree $\implies |E'| = |V| - 1$ by the previous property $\implies E = E' \implies$ no edges were removed $\implies G$ was acyclic already $\implies G$ is a tree.

> An undirected graph is a tree $\iff$ there exists a unique path between any pair of nodes

Otherwise, there must be a cycle.

5.1.1 Greedy Approach

Just repeatedly add the next lightest edge that doesn't produce a *cycle*. The correctness of this algorithm relies on the **cut property**.

5.1.2 Cut Property

> **Definition.** Suppose edges X are part of a minimum spanning tree of $G = (V, E)$. Pick a subset of nodes S such that no edges in X cross between S and $V - S$. Let e be the lightest edge across this partition. Then, $X \cup \{e\}$ is part of some MST.

Proof. If e is part of X's MST, T, then we are done. Thus, assume henceforth that $e \notin T$. Add edge e to T. Since T is connected, this must have generated a cycle since there was already a path between the endpoints of e. This cycle must therefore have some edge across the cut $(S, V - S)$. Remove this edge to produce $T' = T \cup \{e\} - \{e'\}$. T' is connected, since

> Removing a *cycle edge* cannot disconnect a graph.

By tree properties, it is also a tree. Consider the weight of T'.

$$\text{weight}(T') = \text{weight}(T) + w(e) - w(e')$$

Since e is the lightest crossing edge, $w(e) \leq w(e')$. Thus, $\text{weight}(T') \leq \text{weight}(T) \implies T'$ is an MST.

5.1.3 Kruskal's Algorithm

The aforementioned greedy approach is Kruskal's Algorithm. Consider any edge we add in this process. It's the lightest edge we have yet to add, by definition, and since it does not generate a cycle, it satisfies the cut property. Therefore, this edge is part of some MST, and the end result of the algorithm will be an MST.

Kruskal itself is implemented via a *disjoint set union*. This data structure efficiently merges disjoint sets of nodes and checks for cycles (whether or not two nodes are in the same disjoint set).

5.1.4 Disjoint Set Union

We store each set as a tree with a root node / representative. When checking if two nodes are in the same set, we simply check if they have the same root node. (find).

When merging sets, the naive way is to just choose one of the sets' root nodes to be the new root node at random; a smarter way is to choose it based on **rank** or **size**. We'll consider rank. In essence, the root node of the set with the higher rank is made the new root node of both. That is, the shorter tree's root node points to the root of the taller tree. (union). This gives a worst case complexity for find and union of $O(\log n)$, whereas the naive solution is $O(n)$.

Proof.

> **Property 1**. For any x, $\text{rank}(x) < \text{rank}(\text{parent}(x))$.

A root node of rank k is created by the merger of two trees with roots of rank $k-1$. It follows by induction that

> **Property 2**. Any root node of rank k has $\geq 2^k$ nodes in its tree.

This extends to internal nodes as well, actually. Importantly, different rank-k nodes cannot have common descendants, which means that

> **Property 3**. If there are n elements overall, there can be at most $\frac{n}{2^k}$ nodes of rank k.

That is, the maximum rank is $\log n$. This allows the implementation of find and union to have $O(\log n)$ time complexity.

Path Compression

With the above data structure, Kruskal's Algorithm has a runtime of $O(|E|\log|V|)$ for sorting the edges (since $\log|E| \approx \log|V|$) and a runtime of $O(|E|\log|V|)$ for DSU operations. However, what if the edges were already sorted? Then the DSU becomes the bottleneck.

We can actually develop more efficient operations for DSUs. Every time find is called, we do **path compression**. Since find is a recursive function that visits the parent until reaching the root, on its way back through the recursive stack, it can set the parent of each node directly to the root. This results in the *amortized cost* of the find operation becoming $O(1)$, since the distance between a node and its root will be reduced to 1 every time find passes through that node. union also becomes $O(1)$ by extension. The formal proof of this will not be discussed here, though.

5.1.5 Prim's Algorithm

An alternative to Kruskal's Algorithm is Prim's Algorithm, which functions identically to Dijkstra's Algorithm except it tracks the minimum distance from the MST discovered so far for each node. It maintains a priority queue ordered by the current distance of each (MST-adjacent) node from the current MST, and extends the MST by one edge each time by popping a node of the priority queue, adding the corresponding cost-minimizing edge that connects this node, and relaxing all adjacent vertices to this node that are not currently in the MST.

Aside: Randomized Algorithm for Minimum Cut

Consider the last edge that Kruskal's Algorithm adds to the MST of an unweighted graph. This forms a cut $(S, \overline{S})$ in the graph. Remarkably, there is a probability $\geq \frac{1}{n^2}$ such that $(S, \overline{S})$ is the minimum cut in the graph, where the size of the cut $(S, \overline{S})$ is the number of edges cross between S and $\overline{S}$. In other words, repeating the process $O(n^2)$ times and outputting the smallest cut found is likely to identify the minimum cut in G. This thus reveals an $O(mn^2 \log n)$ algorithm for unweighted minimum cuts. (And a bit of further optimization provides an $O(n^2 \log n)$ algorithm).

Why does this work? At each stage of Kruskal's algorithm, the vertex set V is partitioned into connected components. The only edges that can be added to the MST must be connecting two distinct components. Meanwhile, the number of edges incident to each component must be at least the size of the minimum cut in G. Let this be C. If there are k components in the graph, the number of eligible edges is $\geq \frac{kC}{2}$. (Each component has at least C incident edges, divide by 2 for overcounting). The edges are randomly ordered since the graph is unweighted; therefore, the chance that the next eligible edge chosen is from the minimum cut is at most

$C/\left(\frac{kC}{2}\right) = \frac{2}{k}$. Therefore, with probability $1 - \frac{2}{k} = \frac{k-2}{k}$, the minimum cut is left intact. Extending this to all iterations of Kruskal's, we get

$$\frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{2}{4} \cdot \frac{1}{3} = \frac{1}{n(n-1)} \approx \frac{1}{n^2}$$

5.2 Huffman Codes

Input is a text of length T from an alphabet Γ with n letters and frequencies $\{f_i : i \in \Gamma\}$. Output is a cost-minimizing encoding of the alphabet for this text, given that

$$\text{cost}(T) = \sum_i (f_i)(\#\text{ of bits in encoding of } i)$$

In order to ensure decodability, we require that an encoding cannot be a prefix of another encoding. We do this by creating a binary tree, in which left edges represent a 0 and right edges represent a 1, and only assigning leaves to be encodings.

Algorithm.
Sort symbols by ascending order of frequency. Associate each of these symbols with a node. Pop the first two symbols f_i, f_j (lowest frequencies). Connect these two with a parent node, and push a symbol (either one, doesn't matter) with frequency $f_i + f_j$. This corresponds to the parent node.

```
pq = priority_queue<freq, sym>;
for (int i = 1; i < n; i++) {
    pq.push(<freq[i], i>);
    // create node for i
}
for (int i = n; i < 2n - 1; i++) {
    a = pq.pop(); b = pq.pop();
    // create node for i
    pq.push(freq[a] + freq[b], i);
}
```

Proof.

> Claim: there exists an optimal tree T where the lowest two frequencies f_i, f_j are the deepest siblings.

Let O be an optimal tree. Let f_u be the deepest node in O; f_u must have a sibling f_v. Let f_i, f_j exist somewhere in the tree. Now, swap f_i with f_u and f_j with f_v to produce a new tree M. We note that $f_i, f_j \leq f_u, f_v$. Therefore, $\text{cost}(M) \leq \text{cost}(O) \implies \text{cost}(M) = \text{cost}(O)$. Therefore, M must be optimal, and the claim is proved true.

> Lemma: Huffman finds the optimal tree

Base Case: $n = 2$
The tree encodes one symbol to 1 and the other to 0. This is clearly optimal.

Inductive Hypothesis: $n = k$
Assume that the lemma is true for $n = k$.

Inductive Step: $n = k + 1$
By the previous claim, we know there exists an optimal solution O_{k+1} with f_i, f_j as the deepest nodes. Note that

$$\text{cost}(O_{k+1}) = \text{cost}(O_k) + f_i + f_j$$

Where O_n is the tree formed after removing f_i, f_j but keeping $f_i + f_j$ as the new leaf node. This equality makes sense because adding back f_i, f_j will increase the cost of the subtree rooted at $f_i + f_j$ from $(f_i + f_j)x$ to $f_i(x + 1) + f_j(x + 1)$, where x is the depth of the $f_i + f_j$ node.

Recall that Huffman will place f_i, f_j as siblings in the tree. Running Huffman on the smaller subproblem of size k (removing f_i, f_j but keeping $f_i + f_j$ as a new symbol) will, by the induction hypothesis, generate an optimal tree. Let H_k represent the Huffman-generated tree of size k. Then, we can write the following equalities:

$$\begin{aligned}\text{cost}(H_k) &= \text{cost}(O_k) \\ \text{cost}(H_{k+1}) &= \text{cost}(H_k) + f_i + f_j\end{aligned}$$

The first is based on the fact that the cost of H_k is optimal, and the second is based on the aforementioned construction and extension to $k + 1$. We can then write

$$\begin{aligned}\text{cost}(H_{k+1}) &= \text{cost}(O_k) + f_i + f_j \\ &= \text{cost}(O_{k+1})\end{aligned}$$

In other words, the solution that Huffman encoding produces is optimal.

□

5.3 Horn Formulas

Horn formulas are a methodology expressing logical facts and deriving conclusions. Knowledge is represented by

- *Implications*, of the form $(x_1 \wedge x_2 \wedge \cdots \wedge x_n) \implies y$.
- *Pure negative clauses*, of the form $(\overline{x_1} \vee \overline{x_2} \vee \cdots \vee \overline{x_n})$.

Given these clauses, the goal is determining if there is an assignment of variables such that all clauses are true—this is a **satisfying assignment**. This can solved with a greedy algorithm

1. Set all variables to false
2. While there is an implication with an empty left side, set the variable on the right side to true and remove it from the left side of all other implications.
3. Finally, if all pure negative clauses are satisfied, the assignment has been found; otherwise, the formulas are not satisfiable.

That is, this algorithm greedily chooses to guarantee satisfaction of all negative clauses at first. Then, it only sets variables to true if it's necessary to satisfy an implication.

5.4 Set Cover

Consider an undirected graph $G = (V, E)$. A set cover problem involves finding the set of vertices of minimum size such that, $\forall v \in V$, v is in the set or at most one edge away from a vertex in the set.

A greedy solution that immediately comes to mind is to simply add the node that would cover the most new nodes. However, this solution does not necessarily produce the most optimal set cover! But, it is very close to optimal.

> **Claim.** Suppose B contains n elements and that the optimal cover consists of k sets. Then the greedy algorithm will use at most $k \ln n$ sets.

Note that, here, each set is one of the chosen nodes, as described above, along with the new nodes it covers.

Proof. Let n_t be the number of elements not covered after t iterations of the greedy algorithm. Given that the optimal cover is k sets, there must exist some set in the optimal cover that has at least n_t/k of the remaining elements. Thus,

$$n_{t+1} \leq n_t - \frac{n_t}{k} = n_t \left(1 - \frac{1}{k}\right)$$

This implies that

$$n_t \leq n_0 \left(1 - \frac{1}{k}\right)^t$$

Since $1 - x \leq e^{-x}$, with equality only when $x = 0$, we can write

$$n_t \leq n_0 \left(1 - \frac{1}{k}\right)^t < n_0 (e^{-1/k})^t = n e^{-t/k} \implies \boxed{t = k \ln n}$$

As, at $t = k \ln n$, $n_t < n e^{-\ln n} = 1$, i.e. all elements are covered. Therefore, the ratio between the greedy algorithm's solution and the optimal solution is at most $\ln n$. This is the **approximation factor** of the greedy algorithm. In fact, this is enough; there is, in fact, no provably polynomial time algorithm with a smaller approximation factor.

6. Dynamic Programming

6.1 Shortest paths in DAGs

Dynamic programming solutions are, implicitly, taking advantage of the linearization of a hidden DAG.

6.2 Longest Increasing Subsequences (LIS)

Naive DP

Memoize the length of the LIS ending at each index. Array is already a linearized DAG as is! Iterate forward through array, and find the longest LIS among previous indices such that this element could extend the LIS.

```
int n;
vector<int> v(n);
vector<int> dp(n);

for (int i = 0; i < n; i++) {
    dp[i] = 1;
    for (int j = 0; j < i; j++) {
        if (v[i] > v[j])
            dp[i] = max(dp[i], dp[j] + 1);
    }
}
```

It's also possible to maintain an array to track ancestors, to recover the LIS. This has a time complexity of $O(n^2)$.

Optimized DP

We can improve upon the previous algorithm. The idea behind this algorithm is that we only really need to maintain one "candidate LIS." In essence, we maintain a DP array that tracks the LIS, except, for each iteration (iterating forward), we either (1) append to the end of the candidate LIS or (2) replace an element in the LIS by binary searching for the smallest element in the LIS that is $\geq$ the current element. This candidate LIS does not really track the actual elements of the LIS, as one might notice. However, it does efficiently calculate the length of the LIS.

It's difficult to reason about why this works. The main intuition is that, for each iteration, all elements in the candidate LIS are elements before the current element in the array. And, critically, each element essentially records the LIS ending at that element. Thus, replacing an element in the candidate LIS with the current element promises to potentially lead to an LIS that is *at least as good* as the current one. Let the element x at index i in the candidate LIS, and let the current element under consideration be y. Let y be replacing x in the candidate LIS. By definition, $y \leq x$ and y is greater than the element before x in the candidate LIS. Consider an LIS of which x is a member of, but not the end of. Then, the information of this LIS is already tracked by a further element in the candidate LIS. Now, consider an LIS of which x is the end of. note that any future LIS constructed by extending this LIS must use elements later than y in the array; thus, it is clearly optimal to *greedily* choose to replace x with y in the candidate LIS, as y can only ever lead to a better LIS than x.

C++ code:

```
int n = arr.size();
vector<int> dp;
vector<int> arr(n);

dp.push_back(arr[0]);
for (int i = 1; i < n; i++) {
    if (arr[i] > dp.back()) {
        // new longest sequence
        dp.push_back(arr[i]);
    } else {
        int x = lower_bound(dp.begin(), dp.end(), arr[i]) - dp.begin();
        dp[x] = arr[i];
    }
}
return dp.size();
```

6.3 Edit Distance

Given two strings, the algorithm can perform 3 operations:

1. Insert a character into one of the strings
2. Delete a character from a string
3. Substitute a character in a string with another character

The goal is to find the minimum number of operations to make the two strings equal; i.e., the edit distance.

We can solve this problem with a 2D dynamic programming approach. Let i denote the current index in s_1 and j denote the current index in s_2, where s_1, s_2 are the input strings. Let $dp[i][j]$ denote the edit distance of the two strings $s_1[0:i]$ and $s_2[0:j]$. Then we can note that $dp[i][j]$ is equivalent to the minimum of the following subproblems:

1. $dp[i][j-1]+1$, i.e. the equivalent of adding a character to $s_2[0\dots(j-1)]$ or deleting a character from $s_1[0\dots i]$ after equalizing the two strings.
2. $dp[i-1][j]+1$, i.e. the same as (1) but the two strings are switched.
3. $dp[i-1][j-1]+1$ if $s_1[i] \neq s_2[j]$, i.e. substituting $s_1[i]$ with $s_2[j]$, or vice versa.
4. $dp[i-1][j-1]$ if $s[i] = s_2[j]$, i.e. the cost of equalizing $s_1[0\dots(i-1)]$ and $s_2[0\dots(j-1)]$, since $s_1[i]$ is already equal to $s_2[j]$.

Therefore, it suffices to construct a DP table for this problem. This clearly is $O(mn)$ in time complexity.

6.4 Knapsack

Problem: Given n items with a weight and value (w, v) and a total weight capacity W, determine the maximum value V that can be achieved without exceeding W.

With Repetition

In this version of the problem, it assumes there is an infinite number of each item, i.e. the same item can be chosen multiple times.

Let $V(x)$ denote the maximum value V that can be achieved without exceeding the capacity x. Let (w_i, v_i) denote the weight and value of item i. Let I denote the set of items. Then, it follows that

$$V(x) = \min(\{V(x-w_i)+v_i,\ \forall i \in I : w_i \leq x\})$$

In other words, at each capacity, simply consider the addition of each item to a subproblem whose capacity is reduced by that item's weight. Hopefully this makes sense intuitively!

Of course, recursively finding this is computationally expensive. Instead, we memoize our results while iterating forwards through the possible capacities $\{1,\dots,W\}$. Additionally, at each step, it's necessary to check n subproblems. Thus, the total time complexity is $O(nW)$.

Without Repetition

In this version, it assumes there is exactly one instance of each item, i.e. an item can only be chosen once.

With this problem, the strategy is somewhat similar. However, we reverse the order. Instead of iterating through the set of capacities $\{1, \ldots, W\}$ and iterating through all items for each capacity, we iterate through all items and iterate through the set of capacities for each item. Additionally, we iterate the set of capacities in reverse, starting from W. This change helps maintain the requirement that, for every member of the memoized DP array, it will only have considered each element once. (Iterating forward the capacities result in $V(x)$ being reused later for $V(x + w_i)$).

Thus, since the iterations are essentially the same, the time complexity of this algorithm is $O(nW)$ too.

6.5 Chain Matrix Multiplication

There are, in fact, different costs to the order in which one multiplies a sequence of matrices $A_1 \times \cdots \times A_n$. (Where one chooses to place parentheses, since matrix multiplication is *associative*). (Also, recall that the sizes of the matrices must be $a_1 \times a_2$, $a_2 \times a_3$, ..., $a_n \times a_{n+1}$, resulting in the different costs of each matrix multiplication).

The greedy approach, to always perform the cheapest matrix, fails. We consider a dynamic programming approach instead. Note first that the cost of multiplying an $m \times n$ matrix by a $n \times p$ matrix is approximately $O(mnp)$.

Now, think about the problem like divide and conquer first. Our problem is multiplying $A_1 \times \cdots \times A_n$. In divide and conquer, we might choose a matrix A_k, $1 \leq k < n$, such that we then compute the problems $A_1 \times \cdots \times A_k$ and $A_{k+1} \times \cdots \times A_n$ separately, and then combining afterwards. However, we must also consider all possible k, as it is not guaranteed that our split is optimal.

To write out the subproblems, we first generalize this conclusion by replacing A_1 with A_i and A_n with A_j, just to make notation easier. Then, let $C(i, j)$ denote the cost of multiplying $A_i \times \cdots \times A_j$, where $1 \leq i \leq j \leq n$, with an obvious base case of $C(x, x) = 0$. Then,

$$C(i, j) = \min_{i \leq k < j} \{C(i, k) + C(k + 1, j) + a_{i-1} \cdot a_k \cdot a_j\}$$

This is sufficient to solve the problem, and we simply compute this for all i, j, memoizing to avoid recomputation. This is a 2D table of size n^2, and calculating each entry requires looping through, at most, $O(n)$ other entries. Thus, the total runtime is $O(n^3)$.

6.6 Shortest Paths

Shortest Reliable Paths

The shortest reliable path is defined as the shortest path from s to t that uses at most k edges.

Let $dist(v, i)$ be the length of the shortest path from s to v that uses i edges, where $i \leq k$. Let $\ell(u, v)$ denote the weight of the edge from u to v. We can then define

$$dist(v, i) = \min_{(u,v) \in E} (dist(u, i-1) + \ell(u, v))$$

Thus, we have a DP relation. We can then essentially run a modified version of Bellman-Ford's, where we iterate k times and maintain a 2D DP array $dist$ of size $k \times n$. Since we need to check each edge at most twice for each iteration, the total runtime is $O(km)$.

All-Pairs Shortest Paths

The **Floyd-Warshall** algorithm finds the shortest path between **all pairs** of vertices in a graph, and works even in the presence of negative edges. The pseudocode is as follows:

```
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}
```

Source

In essence, Floyd-Warshall's maintains a 2-d matrix d that keeps track of the distance from node i and node j in $d[i][j]$. Before the kth step, $d[i][j]$ stores the minimum distance path whose intermediate nodes are only within the set $\{1, \ldots, k-1\}$. At each step, either $d[i][j]$ is still the minimum or, if $d[i][k]$ and $d[k][j]$ has been found, then there may be a new minimum path traversing through node k.

The runtime of Floyd-Warshall's is clearly $O(n^3)$.

Traveling Salesman Problem

The Traveling Salesman Problem (TSP) is a known NP-hard problem. Essentially, given a graph of n nodes and pairwise distances between nodes, it asks for the best order of traversing the nodes starting from some node v such that each node is visited exactly once and the path begins and ends at v. In essence, the minimum cost Hamiltonian tour.

The brute force approach evaluates every single tour, which gives a time complexity of $O(n!)$. With dynamic programming, it's possible to reduce this to exponential time complexity. Consider how we might construct a subproblem. An intuitive idea is to consider the subproblem of a prefix of the tour, i.e. a path $v, \ldots, u$ that does not fully complete the tour. More formally, for a subset of nodes $S \subseteq \{1, 2, \ldots, n\}$ that includes 1 and the node $j \in S$, let $C(S, j)$ be the length of the shortest path visiting each node in S exactly once, starting at 1 and ending at j. In essence, a Hamiltonian path for a subset of the graph. Note that $C(S, 1) = \infty$ since the path should not start and end at 1 unless S is precisely the entire set of nodes.

Now, we calculate $C(S, j)$ based on its subproblems. This involves essentially just iterating through all possible choices for the second-to-last node. In other words,

$$C(S, j) = \min_{i \in S, i \neq j} C(S - \{j\}, i) + dist(i, j)$$

There are at most $2^n \cdot n$ subproblems, and each takes linear time to combine. Thus, the total runtime is $O(n^2 2^n)$.

6.7 Independent Sets in Trees

We denote a subset of nodes $S \subset V$ as an *independent set* of a graph $G = (V, E)$ if there are no edges between any nodes in S. One problem is to find the largest independent set of a graph. This problem is, in general, intractable. However, in a tree, the problem can be solved in *linear* time using dynamic programming.

Let $dp(v)$ denote the size of the largest independent set of the subtree rooted at v. We consider the possible subproblems. Assume first that we include the root node in the set. Then, we cannot include any of the children of the root node. However, we can include the union of the largest independent sets of each of its grandchildren. Assume next that we don't include the root node. Then, we can include the union of the largest independent sets of each of its children. Therefore, we have the following dp relation

$$dp(v) = \max \left(1 + \sum_{\text{grandchildren } w \text{ of } u} dp(w), \sum_{\text{children } w \text{ of } u} dp(w) \right)$$

Thus, we can root the node at an arbitrary node r and simply perform DFS with memoization (calculating in the postorder routine). Thus, the overall runtime is $O(n + m)$.

7. Linear Programming and Reductions

7.1 Introduction

For a linear programming problem, we are provided a set of n variables, to which we must assign real values such that they

1. Satisfy a provided set of linear equations/inequalities
2. Maximize/minimize a given linear objective function

We can draw the linear constraints on an n-dimensional hyperplane. The optimum solution, if it exists, will be located at a vertex of the n-dimensional polygon. This should naturally make sense; the optimum should make at least one of the inequalities reach its extremity.

Occasionally, the linear program can have no optimum, though. This occurs when the linear program is *infeasible*, i.e. the constraints are unsatisfiable, or *unbounded*, when it is possible to achieve arbitrarily small/large objective values.

Solving Linear Programs

All linear programs can be solved by the ***simplex method***. Essentially, this algorithm starts at an arbitrary vertex, and greedily traverses to an adjacent vertex with a better objective value, until it reaches a vertex whose direct neighbors all have worse or equal objective value. Essentially, it *hill-climbs* up to the optimal vertex.

Why does local optimality imply global optimality? That is, why is the greedy algorithm correct? Consider the profit line passing through the vertex. Since all the vertex's neighbors lie below this line, the rest of the feasible polygon must also lie below this line **because the polygon is convex.** Simply consider the intersection of two inequalities in a 2D plane. The result must be a convex vertex.

> **Convexity**
> The notion of a "convex" polygon in terms of its angles does not extend well to higher dimensions. However, the formal definition of convexity extends to higher dimensions: a convex region is a set of points S such that $\forall x, y \in S$, the line segment $x - y$ is entirely contained within the region.

Notably, all linear programs (LPs) can be *reduced to one another* via simple transformations. This is intuitively evidenced by how all LPs can be solved by the simplex algorithm.

Variants of LP

A general linear program has several degrees of freedom

1. It can be either a maximization or minimization problem
2. Its constraints can be equations and/or inequalities
3. The variables are often restricted to be nonnegative, but they can also be unrestricted in sign

Additionally, we discuss the simple transformations that can reduce all LP options to one another.

1. To transform a maximization problem to a minimization problem, or vice versa, negate all coefficients of the objective function.
2. 1. To turn an inequality constraint $\sum_{i=1}^{n} a_i x_i \leq b$ into an equation, we introduce a new variable s and replace the inequality with the equation and inequality pair $\sum_{i=1}^{n} a_i x_i + s = b$ and $s \geq 0$. This is called the *slack variable* for the inequality.
   2. To change an equality into an inequality, simple rewrite $ax = b$ into the pair of inequalities $ax \geq b$ and $ax \leq b$.
3. To transform a variable x unrestricted in sign, we introduce two nonnegative variables $x^+, x^- \geq 0$ and replace x with $x^+ - x^-$. Thus, x can take on any real value by adjusting these variables.

Applying these transformations enables us to reduce any LP to the *standard form*, which are described by the following constraints.

1. All variables are nonnegative
2. All constraints are equations
3. The objective function should be minimized

7.2 Flows in Networks

Maximum Flow

In this problem, we are provided a directed/undirected graph G in which each edge has a capacity c_e. The goal is to find the *maximum flow* that can be sent from a source node s to sink node t such that the capacities of each edge are not exceeded. More formally, we define a flow within a network to be a set of variables f_e for each edge e in the network, satisfying the following properties.

1. It doesn't violate edge capacities, i.e. $0 \leq f_e \leq c_e$, $\forall e \in E$.
2. For all nodes u besides s and t, the amount of flow entering u equals the amount of flow exiting u. That is, flow is *conserved*.

Then, the *size* of a flow is the total quantity sent from s to t, which, by the conservation principle, is the quantity of flow leaving s. The maximum possible size across all valid flows is the maximum flow.

Notice how these constraints are just a linear program with several equations and an objective function (amount of flow leaving s) to be maximized.

Simplex Solution

Since maximum flow is an LP, we can apply the simple simplex solution to this problem! Of course, at the moment, we really only have a geometric intuition regarding the notion of an optimal solution. We won't formalize the generalized simplex algorithm yet. But, it turns out the maximum flow problem has a fairly concise interpretation of the simplex algorithm.

1. Start with zero flow.
2. Repeatedly choose an appropriate path from s to t, known as an **augmenting path**, and increase the flow along the edges of this path as much as possible.*

As you may notice, this algorithm seems remarkably like a greedy algorithm! This should make sense, intuitively. Recall that the simplex algorithm is really just a greedy hill-climbing algorithm in the geometric interpretation.

You might notice that * for the second step though. There's a slight complication with a method; this greedy design is actually *not always optimal*, and is not the simplex algorithm for maximum flow. Instead, the simplex algorithm introduces one detail that makes it optimal: it allows new paths to *cancel existing flows*.

Thus, the simplex algorithm, for each iteration, finds a path from $s \to t$ whose edges (u, v) can be categorized as one of two types.

1. (u, v) in the original network, and is not yet at full capacity. Then, this edge can handle up to $c_{uv} - f_{uv}$ additional units of flow.
2. The reverse edge (v, u) is in the original network, and there is some flow along it. Then, this edge can handle up to f_{vu} additional units of flow.

These opportunities to increase flow can be imagined as a slightly modified graph G^f, known as a *residual network*, which represent the remaining flow that can be added along an edge. Thus, simplex is really just choosing an $s \to t$ path in this residual network.

> Note that this algorithm is actually known as the **Ford-Fulkerson method**.

Minimum Cut

Maximum flow is actually the **dual** of another graph problem, *minimum cut*. The minimum cut problem presents the same graph as a maximum flow problem, except the goal is not to maximize flow, but minimize the total capacity of all edges from L to R, where L, R are two disjoint groups of vertices such that $s \in L$ and $t \in R$. This is known as an (s,t)-*cut*; thus, the minimum cut problem seeks to minimize the "size" of the (s,t)-cut.

> **Dual?**
> For an LP problem, its dual is a problem with equivalent solution, but phrased as the reverse optimization for the objective function. For maximum flow, it seeks to find the *maximum* size of a flow from s to t. For minimum cut, its dual, it seeks to find the *minimum* total capacity across an (s,t)-cut. It's that simple!

In fact, this gives rise to a theorem known as

> **Max-Flow Min-Cut Theorem**
> The size of the maximum flow in a network equals the capacity of the smallest (s,t)-cut.

Efficiency

Though we have outlined the simplex algorithm for maximum flow, we have yet to discuss exactly how efficient it is.

Certainly, we can run a DFS or BFS algorithm on the residual graph for each iteration to find a valid path. But, how many times will this repeat? If we suppose all edges in the original network have *integer* capacities $\leq C$, then we can prove via induction that each iteration will increase the flow by an integer amount (≥ 1). This is actually known as the

> **Integral Flow Theorem**
> If every capacity in the network is an integer, then the size of the maximum flow is an integer, and there exists a maximum flow such that each individual edge flow is integral.

As a direct result of this theorem, we can show that the algorithm will take at most C iterations, and with a runtime of $|E|$ from DFS/BFS, the total runtime is $O(C|E|)$. But, we can do better!

Indeed, if the flows are chosen poorly, the simplex algorithm can take $O(C|E|)$ steps. However, it turns out using BFS to choose the flow each iteration actually bounds the number of iterations by $O(|V||E|)$. Thus, the total runtime is $O(|V||E|^2)$.

Edmonds-Karp Algorithm

The Edmonds-Karp algorithm, as described in the previous section, is just the simplex algorithm with BFS applied to the residual graph each iteration to identify a new flow. The key idea of using BFS is that, every time an augmenting path is found, one of the edges will become saturated. This edge may be revisited in the future, but, since BFS will always find paths in increasing order of edges, the number of edges between s and this edge will increase by at least *one* each time it is revisited in an augmenting path. An augmenting path must, before visiting this edge, traverse at least one vertex, s, and can traverse at most $|V| - 1$ vertices (all vertices except t). Therefore, this edge will reappear in an augmenting path at most $O(|V|)$ times. Thus, considering all edges, the number of augmenting paths that will be found by this algorithm is bounded by $O(|V||E|)$. Since BFS has a runtime of $O(|E|)$, the total runtime is $O(|V||E|^2)$.

□

As a bonus, the Edmond-Karp algorithm works for even graphs with irrational edge capacities.

7.3 Bipartite Matching

Consider a bipartite graph whose two sets are equally sized and denoted A and B. The **Bipartite Matching** problem considers if it's possible to form a matching of pairs between nodes in A and B such that every pair of nodes has an edge between them. This is known as a *perfect matching*, in graph theoretic terminology.

This problem can actually be reduced to a maximum flow problem! Create a source node s with outgoing edges to all nodes in A and a new sink node t with incoming edges from all nodes in B. Then, assign each edge a capacity of 1. Then, there exists a perfect matching $\iff$ the network has a flow whose size equals the number of pairs (which is the same as $|A|$ or $|B|$). There is one nuance to this reduction, though; the *Integral Flow Theorem* guarantees that, if the network has a maximal flow satisfying the above condition, then there exists a valid perfect matching since each individual edge flow is integral.

7.4 Duality

Primal LP:

$$\begin{aligned} &\max\ c_1x_1 + \cdots + c_nx_n \\ &a_{i1}x_1 + \cdots + a_{in}x_n \le b_i \quad \text{for } i \in I \\ &a_{i1}x_1 + \cdots + a_{in}x_n = b_i \quad \text{for } i \in E \\ &x_j \ge 0 \quad \text{for } j \in N \end{aligned}$$

Dual LP:

$$\begin{aligned} &\min\ b_1y_1 + \cdots + b_my_m \\ &a_{1j}y_1 + \cdots + a_{mj}y_m \ge c_j \quad \text{for } j \in N \\ &a_{1j}y_1 + \cdots + a_{mj}y_m = c_j \quad \text{for } j \notin N \\ &y_i \ge 0 \quad \text{for } i \in I \end{aligned}$$

7.5 Zero-Sum Games

A zero-sum game is a game with one or more players in which the expected payout for each player is 0. For instance, rock-paper-scissors (RPS) is a simple example of a zero-sum game.

For a zero-sum game, we can actually represent the situation with an LP. Consider two players A and B playing RPS. Each player gains 1 point for a win, -1 for a loss, and 0 for a draw. Both players, of course, seek to maximize their score. In particular, each player's goal also implies that they seek to minimize their opponent's score. WLOG, let us focus on A's score. Then, A will play to maximize A's score, and B will play to minimize A's score.W

Well, if A chooses to play completely randomly, then it turns out that B can force an expected score of 0 for A, no matter what A does. Conversely, if B chooses to play completely randomly, then it turns out that A can force an expected score of 0 for B. In other words, neither player can hope to increase/decrease the expected score of A beyond 0. Thus, the best each player can do is play randomly, with an expected score of 0.

This remains the same even if one of A or B announces their strategy first! WLOG, let A announce their strategy first. Intuitively, one would expect this to favor B, then. However, if both players play optimally (i.e. A chooses the random strategy), B cannot take advantage of this, and thus the expected score is still 0. Remarkably, this is a consequence of linear programming duality, as you may have noticed from the maximization/minimization parallel. In essence, A, in their choice of strategy, is maximizing the minimum guaranteed expected average for themselves, while B is minimizing the maximum guaranteed average of A's score. In other words, A is choosing the strategy such that it guarantees an expected score $\geq x$, where x is maximal, regardless of B's choice of strategy. B is choosing a strategy such that it guarantees A's expected score is $\leq y$, where y is minimal, regardless of A's choice of strategy.

Interestingly enough, in zero-sum games, $x = y$, i.e. there is a uniquely defined optimal strategy (across both players). This can be generalized to arbitrary games, and essentially illustrates the existence of mixed strategies that are optimal for both players and achieve the same value. This result is known as the **minimax theorem**, and is described by the equation

$$\max_{\mathbf{x}} \min_{\mathbf{y}} \sum_{i,j} G_{ij} x_i y_i = \min_{\mathbf{y}} \max_{\mathbf{x}} \sum_{i,j} G_{ij} x_i y_i$$

Note that when writing the LP for, WLOG, player A, one should seek to maximize some variable a with constraints describing the idea that, no matter what strategy player B picks, A 's expected score is $\geq a$. For instance, consider the zero-sum game described by the following matrix

$$\begin{bmatrix} & x & y & z \\ x & -2 & 1 & 1 \\ y & 0 & -2 & 0 \\ z & 3 & 3 & -2 \end{bmatrix}$$

Where the rows describe A's choices and the columns describe B's choices, and M_{ij} describe the score A receives if A picks choice i and B picks choice j. Then, our LP has an objective function of $\max a$ and constraints of

$$\begin{aligned} -2x + 3z &\geq a \\ x - 2y + 3z &\geq a \\ x - 2z &\geq a \\ x + y + z &= 1 \end{aligned}$$

Make sure this makes sense to you before moving on :)

> ⚠ **Writing LPs for Zero Sum Games**
>
> This is a bit of tricky topic for me personally, which I've come back to write about while studying for the CS 170 final exam. In short, just remember the following. Player A is always looking to maximize some quantity z, where z is the minimum (bounded by $\leq$ constraints) score that player B is able to induce. This involves writing a $z \leq$ constraint for *each possibility for player B*, to ensure none of the possibilities can allow B to induce a score less than z. The analogue is true for player B, in which they are looking to minimize some quantity bounded by $\geq$ constraints for *each possibility for player A*.

7.6 The Simplex Algorithm

Finally, we describe the generalized simplex algorithm for solving LP problems. At a high level overview, the simplex algorithm receives an input of a set of linear inequalities, a set of variables in those inequalities, and a linear objective function, and finds the optimal feasible point using the follow strategy.

> Let v be any vertex of the feasible region. While there exists a neighbor v' of v with better objective value, $v \leftarrow v'$.

This is easy to visualize in 2D or 3D. But, what about in n dimensions?

Notation

First, it's necessary to clarify some notations. Let $x_i \in \mathbb{R}$ represent the variables, where $1 \leq i \leq n$. Any linear equation involving the x_i's defines a *hyperplane* in the space $\mathbb{R}^n$. A linear

inequality defines a *half-space*, all points that are either precisely on the hyperplane or lie on one side of it.

Meanwhile, a *vertex* is a unique point at which some subset of hyperplanes meet. This is always specified by at least n inequalities. In particular, note that, with more than n equations, we are guaranteed linear dependency; in other words, each vertex is actually specified by a set of precisely n inequalities. By extension, we can note that two vertices are *neighbors* if they share $n-1$ defining inequalities.

*Notably, there can exist vertices that are generated by two different sets of inequalities. In particular, this occurs whenever some we have linear dependency for vertex. (which implies there are multiple sets of n inequalities that can generate this vertex). These are *degenerate* vertices, and we will discuss how to handle them later. For now, assume all vertices are nondegenerate.

Algorithm

The algorithm, as previously described, consists of two simple steps.

1. Check the optimality of the current vertex, and halt if optimal.
2. Determine where to move next. Repeat.

Origin

Both steps are, in fact, easy if the vertex is at the origin. Consider a generic LP

$$\begin{aligned} &\max \mathbf{c}^T x \\ &\mathbf{A}\mathbf{x} \leq \mathbf{b} \\ &\mathbf{x} \geq \mathbf{0} \end{aligned}$$

Where $\mathbf{x} = [x_1, \ldots, x_n]^T$. Also, suppose the origin is within the feasible region. Then, it is certainly a vertex, as it is the unique point at which the n inequalities $\{x_1 \geq 0, \ldots, x_n \geq 0\}$ reach their extremes, or are *tight*.

Task 1:

Claim: the origin is optimal $\iff c_i \leq 0, \forall i$.
Proof: If all $c_i \leq 0$, then, considering that $\mathbf{x} \geq 0$, it is impossible to increase the objective value, since each x_i can only increase, and this will only result in $\mathbf{c}^T x$ decreasing.

□

Task 2:

We can move to another constraint by increasing x_i until another constraint is hit. In other

words, we release the tight constraint of $x_i \geq 0$ (i.e., $x_i = 0$ makes it tight), and thus allow increasing x_i until some other inequality is now tight. This results in precisely n tight inequalities (or more, but recall this can be reduced due to linear dependency).

Non-Origin

But what if we are at a non-origin vertex? Then, it suffices to transform the coordinate system such that this vertex is located at the origin.

Consider the n tight inequalities that determine a non-origin vertex $\mathbf{v}$. Let the equation of one be denoted $\mathbf{a}_i \cdot \mathbf{x} \leq b_i$. Then, the distance from any point $\mathbf{x}$ to that hyperplane is $y_i = b_i - \mathbf{a}_i \cdot x$. These n equations essentially define the distances y_i as linear functions in x_i. The relationship can then be inverted to express the x_i's as linear functions of the y_i's, which consequently allows us to rewrite the entire LP in terms of the y_i's (via substitution). If you consider the hyperplanes as the hyperaxes of the n-dimensional space, then y_i are precisely the coordinates of any solution $\mathbf{x}$ in this new definition of the axes!

Notably, this LP with this new coordinate system has three properties:

1. It includes the inequalities $y \geq 0$, which are just transformed versions of the inequalities defining v's nonnegativity.
2. v is the origin in $\mathbf{y}$-coordinate-space.
3. The cost function is now $\max c_{\mathbf{u}} + \tilde{\mathbf{c}}^T \mathbf{y}$, where $c_{\mathbf{u}}$ is the value of the objective function at $\mathbf{u}$ and $\tilde{\mathbf{c}}$ is the cost function transformed from in terms of x_i to in terms of y_i.

Final Considerations

Starting Vertex

How do we find a starting vertex for the simplex algorithm? This is not entirely trivial because, in general, it's not necessarily true that the origin will be contained within the feasible region. (Otherwise, as previously stated, the origin is always a vertex).

Luckily, finding a starting vertex can itself be reduced to an LP. (And, fortunately, this LP has a known starting vertex).

Consider an LP in standard form, i.e.

$$\min \mathbf{c}^T \mathbf{x} \text{ satisfying } (\mathbf{A}\mathbf{x} = \mathbf{b}) \wedge (\mathbf{x} \geq 0)$$

First, make the RHS of the equations all nonnegative. IF $b_i < 0$, just negate both sides of the i th equation. Then, create the following, separate LP:

- Add m new variables $z_1, \ldots, z_m \geq 0$ such that m is the number of equations.

- Add z_i to the LHS of the ith equation in the original LP to produce m equations for this LP.
- Let the objective to be *minimized* be $z_1 + \cdots + z_m$.

Note that, for this LP, the starting vertex $z_i = b_i,\ \forall i$, and all other variables $(x_1, \ldots, x_n)$ are set to 0. Thus, we can subsequently just start the simplex algorithm.

The result is one of two possibilities. If $\min z_1 + \cdots + z_n = 0$, then $z_i = 0,\ \forall i$, and we get a starting feasible vertex of the original LP by just ignoring the z_i's.

> ⚠ **Why is this true? Make sure it makes sense to you before moving on!**

In contrast, if $\min z_1 + \cdots + z_n > 0$, then the original LP is actually infeasible anyways! In other words, the original LP's equations *required* some nonzero z_i's to be feasible, implying the original LP cannot be solved.

Degeneracy

As aforementioned, we previously assumed all vertices are nondegenerate. But, what if we do have degenerate vertices? Well, it could potentially result in vertices that *seem* like optimal solutions, since they are locally optimal, but are in fact suboptimal.

Naively, we can attempt to fix this by modifying simplex to detect degeneracy and continue moving to a different vertex despite lack of improvements in cost; however, this could result in an infinite loop. Instead, a smarter solution is to *slightly perturb* each b_i to $b_i \pm \epsilon_i$, where ϵ_i is some tiny error value randomly chosen for each i. This ensures that there is no degeneracy in the entire system!

Unboundedness

What if an LP is unbounded in value? That is, the objective function can be made arbitrarily small/large when being minimized/maximized, respectively. In fact, simplex can be modified to detect this. When exploring the neighbors of a vertex, we can detect if removing a tight inequality and replacing it with another results in an undetermined system of equations with infinite solutions. In this case, simplex will simply halt.

Time Complexity

Let the LP have n variables and m inequality constraints. Consider an arbitrary vertex $\mathbf{u}$. By definition it is the unique point at which n inequality constraints are tight. Each of its neighbors must share at least $n - 1$ inequalities with it, and have m degrees of freedom for the other inequality, so $\mathbf{u}$ has $O(mn)$ neighbors.

For each vertex $\mathbf{u}$, we must determine (1) if this vertex is more optimal and (2) if it is a true vertex. The first is simply a dot product. The second task is slow, however; it involves solving a system of n equations in n variables (i.e. satisfying the n chosen inequalities as equations, to ensure they're tight) and checking the feasibility of the result. Applying Gaussian elimination to solve this has a runtime of $O(n^3)$. This results in an overall runtime per iteration of $O(mn^4)$. But, we can do better.

Recall the idea of moving $\mathbf{u}$ to the origin, providing a local view of the neighbors' optimality. It turns out that this transformation only has a runtime of $O((m+n)n)$; this is a result of exploiting the fact that the local view changes by just one defining inequality between iterations. Then, after transforming $\mathbf{u}$ to the origin, recall that the local view of the objective function is $\max c_{\mathbf{u}} + \tilde{\mathbf{c}} \cdot \mathbf{y}$. (Recall that $c_{\mathbf{u}}$ is the value of the objective function at $\mathbf{u}$, and can be calculated with just a dot product). Thus, it actually suffices to pick *any* $\tilde{c}_i > 0$ (if there is none, the current vertex is optimal). And, since the LP has been rewritten in terms of y_i, it's much easier to determine how much y_i can be increased before some inequality is violated. Thus, the total runtime for each iteration of simplex is just $O((m+n)n + mn) = O(mn)$. (Note that $m \geq n$, otherwise the LP is unbounded).

Then, how many iterations of simplex are there? There can't be more than $\binom{m+n}{n}$, since this is the maximum number of possible vertices. This is, in fact, *exponential* in n. And, in fact, it's not possible to produce a non-exponential upper bound; there do exist LPs in which simplex takes an exponential number of iterations! However, this does not typically occur in *practice*.

8. NP-Complete Problems

8.1 Search Problems

Throughout this book (a.k.a. throughout these notes), we have discussed *efficient* algorithms that frequently **search** for a desired solution in polynomial runtime among an *exponential search space*. However, not all such problems can be efficiently solved.

SAT

Consider the SAT or Satisfiability problem. We are provided a Boolean formula in *conjunctive normal form* (**CNF**), which is composed of the logical and (conjunction) of several *clauses*, each the logical or (disjunction) of several *literals*, where a literal is either a Boolean variable or the not of one. For instance, here's an example of a CNF formula.

$$(x \vee y \vee z)(x \vee \overline{y})(y \vee \overline{z})(z \vee \overline{x})(\overline{x} \vee \overline{y} \vee \overline{z})$$

The SAT problem asks us to find an assignment of the Boolean variables such that this statement is true or else report that none exists. This problem, in particular, does not have a solution. But, in general, it's difficult to efficiently find a solution: the search space is 2^n, where n is the number of variables, and there is no known efficient algorithm for the general SAT problem.

Note, though, that the SAT problem is again a typical search problem. Provided an *instance* I (CNF formula), we are asked to find a *solution* S or report that none exists. In particular, a search problem must also have the property that S's validity as a solution for I can be checked in *polynomial runtime*. Formally,

> A *search problem* is specified by an algorithm C that takes two inputs, an instance I and a proposed solution S, and runs in time polynomial in $|I|$. We say S is a solution to $I \iff C(I, S) = \text{true}$

There do, however, exist two notable variants of SAT that can be efficiently solved. If each clause contains at most one positive literal (i.e. x is positive, $\overline{x}$ is negative), then the Boolean formula is a *Horn formula*. We discussed an efficient greedy solution to this problem in Section 5.3. Meanwhile, if all clauses have at most two literals, then the problem is called a 2SAT problem, and it can be solved in linear time by identifying the SCCs of a graph constructed from the instance. (Recall the efficient algorithm we derived for this in Section 3.4).

Conversely, some problems that seem easier than the general SAT problem are still hard to solve! If all clauses have at most three literals, we have what is known as a 3SAT problem—and this problem is actually just as hard as the general SAT problem!

TSP

In the Traveling Salesman Problem, we are provided an instance with n vertices $1, \ldots, n$, all pairwise distances between them, and a budget b. The desired solution is a *tour*, a cycle that passes through each vertex precisely once, of total cost at most b (or else report none).

Typically, TSP is actually discussed as an *optimization* problem, in which the shortest possible tour is desired. However, here we frame it as a *search* problem. Why? So that we can compare TSP with other search problems. Search problems are such a general description of problems that other types of problems can often be framed as search problems, like for TSP. More accurately, in this instance, the optimization version of TSP (which we will call Min-TSP henceforth) **reduces** to TSP. In essence, this means that solving Min-TSP is *at least as hard as* solving TSP. For TSP, this intuitively is because we can find a solution for Min-TSP by binary searching the budget b for TSP. We formalize this notion later.

Also, interestingly enough, one might note that Min-TSP seems like a difficult analogue to the minimum spanning tree problem, which we have developed efficient greedy algorithms for in Section 5.1. In essence, Min-TSP just replaces the minimum spanning tree with a more difficult constraint—a tour. This simple change results in an exponentially harder problem (literally!).

Eulerian and Rudrata/Hamiltonian Paths

Now, we turn to another sibling pair of graph problems.

Several hundred years ago, Euler sought to solve a problem—given a graph G, does there exist a path in the graph such that each edge in the graph is traversed exactly once? It turns out that this problem has an exceedingly elegant solution—there exists a Eulerian path $\iff$ all vertices, except for two (start, end), must have *even degree*.

As before, let's now define this problem as a search problem. The Eulerian path problem provides an instance with a graph G, and the desired solution is a valid Eulerian path. We won't discuss the solution in detail—but, it follows from Euler's observation that the search problem can also be solved in polynomial time.

Before Euler solved his namesake graph problem, though, Rudrata sought to solve another, very similar problem. Given a graph G, does there exist a path in the graph such that each vertex in the graph is traversed exactly once? Again, we frame this problem as a search problem, too. The Rudrata path (also known as Hamiltonian path) problem provides an instance with a graph G, and the desired solution is a valid Rudrata path.

This seems very similar to Euler's problem! Despite this, there is no known efficient algorithm to solve the Rudrata path problem.

> You may have heard the more familiar namesakes Eulerian cycle or Hamiltonian cycle, rather than the path variants described above. Well, as we will see later, these problems are effectively equivalent to their path variants!

Cuts and Bisections

Recall our minimum cut problem from Section 7.2. As aforementioned, a *cut* is a set of edges whose removal leaves a graph disconnected. We can generalize the minimum cut problem to a search problem: given an instance with a graph G and budget b, the desired solution is a cut with at most b edges.

Of course, the minimum cut problem has a polynomial time algorithm. However, there exist other cut problems that, as far as we know, cannot be efficiently solved! In particular, the **Balanced Cut** problem is a variant of the general cut search problem: given an instance with a graph G and budget b, the desired solution is a partition of vertices into sets S, T such that $|S|, |T| \geq \frac{n}{3}$ and there are at most b edges between S and T.

Balanced cuts actually have many important applications, such as *clustering*! However, its intractability forces such applications to use efficient approximations of solutions.

Integer LP

We discussed linear programming extensively in Section 7, and noted that, though the simplex algorithm does not always run in polynomial time, there does exist a polynomial algorithm for linear programming—at least, when the solutions are allowed to be anything in $\mathbb{R}$. When we constrain solutions to be in $\mathbb{Z}$, i.e. *Integer Linear Programming* (**ILP**), the problem becomes intractable!

More formally, let us frame ILP as the following search problem: given an instance of an $m \times n$ matrix $\mathbf{A}$ and an m-vector $\mathbf{b}$, the desired solution is a nonnegative integer vector $\mathbf{x}$ satisfying $\mathbf{Ax} \leq \mathbf{b}$.

There is additionally a special case of ILP that is just as hard, known as *Zero-One Equations* (**ZOE**). In particular, we constrain $\mathbf{A}$ to have entries of only either 0 or 1, $\mathbf{x}$ to be a vector of only 0's and 1's, and $\mathbf{b} = \mathbf{1}$ (the m-vector of all 1's).

Three-Dimensional Matching

Recall the bipartite matching problem we discussed in Section 7.3, which can be efficiently solved with the Edmonds-Karp algorithm. There exists an intractable generalization of this known as **3D Matching**: given an instance with n boys, n girls, and n pets, and compatibilities

among them described by a set of *triples* of the form (b,g,p), the desired solution is a set of n disjoint triples that is a subset of the set of compatible triples.

Independent Set, Vertex Cover, Clique

The **Independent Set** problem: given an instance with a graph G and an integer k, the desired solution is a set of k independent vertices, i.e. no pair of vertices have an edge between them. Recall from Section 6.7 that, via dynamic programming, we could efficiently solve this problem on a tree. However, for general graphs, there exists no known polynomial-time algorithm!

The **Vertex Cover** problem: given an instance with a graph G and budget b, the desired solution is a set of b vertices such that each edge in the graph is adjacent to at least one vertex in this set. This is actually a special case of the **Set Cover** problem: given an instance with a set E, $S_1, \ldots, S_m \subseteq E$, and a budget b, the desired solution is a set of b subsets such that their union is E. Note that we briefly discussed a greedy approximation of a solution to the minimum size Set Cover problem in Section 5.4. Regardless, these problems are both intractable.

There's an interesting parallel between the Independent Set and Vertex Cover problems! Try and see if you can figure it out :)

We'll discuss why this is the case soon... sorry for the suspense :P

3D Matching is also a special case of the Set Cover problem! Try and figure this one out too :>

E is all n boys, n girls, and n pets, the subsets $S_1, \ldots, S_m$ are precisely the triples of (b, g, p), and the budget is n.

Finally, the **Clique** problem: given an instance with a graph G and goal k, the desired solution is a set of k vertices such that all possible edges between them are present (i.e. these vertices form a fully connected subgraph in G).

Longest Path

Recall from Section 4 that the shortest path problem can be solved efficiently using Dijkstra's Algorithm or Bellman-Ford's Algorithm. What about the analogue, **longest path**? Formally, given an instance with a graph G whose edges are assigned nonnegative edge weights, two

vertices s, t, and a goal g, the desired solution is a *simple* path from $s \to t$ with total weight at least g.

It might seem that simply negating all the edge weights, and running a shortest path algorithm that can deal with negative weights on this graph would efficiently solve this problem. However, the issue with this is that these algorithms do not actually find the shortest simple path on a graph with a negative cycle! And, a negative cycle can certainly be created since all edges in the longest path problem are nonnegative (so negating them will make them all nonpositive). And, in fact, there is no known polynomial time algorithm to solve the longest path problem.

Knapsack and Subset Sum

We now frame **knapsack** as a search problem: given an instance with a weight capacity W, a goal g, and a set of n items with weights w_i and vales v_i, the desired solution is a set of items whose total weight is at most W and whose total value is at least g.

In Section 6.4, we discussed algorithms for solving the knapsack problem that run in $O(nW)$ time. Is this polynomial time, though? Well, not in n, since W can be arbitrarily large. If we try and remove W from the time complexity, then it turns out that the best we can do is $O(2^n)$, checking all possible subsets of items. So, there isn't actually a polynomial time algorithm for knapsack!

Actually, we can derive a polynomial time for a somewhat contrived variant: **Unary Knapsack**, in which we represent, e.g., 3 as III. This isn't too useful, but, yes, this does have a polynomial time algorithm, due to the limitation this places on W.

Meanwhile, an equally hard special case of knapsack is also of particular interest. The **Subset Sum** problem: given an instance with n integers x_i and a capacity W, the desired solution is a subset of the provided integers that add up to precisely W. Note that this is a special case of knapsack where $v_i = w_i$ for each item i and $g = W$, and that the constraints $\sum v_i \geq g$ and $\sum w_i \leq W$ in knapsack is equivalent to $\sum w_i = W$ here.

Why is this special case of knapsack interesting, if it's equally as hard as knapsack? The point is simplicity; as we'll soon see, exploring *reductions* between problems is much easier with simpler problems like Subset Sum.

8.2 NP-Complete Problems

Tractability

Consider the following table of sibling hard vs. easy search problems (we'll discuss what NP-Complete and P are soon).

Hard (NP-Complete)	Easy (P)
3SAT	2SAT, Horn SAT
TSP	MST
Longest Path	Shortest Path
3D Matching	Bipartite Matching
Knapsack	Unary Knapsack
Independent Set	Independent Set (Trees)
ILP	LP
Rudrata Path	Eulerian Path
Balanced Cut	Minimum Cut

On the right are easy problems, which are all easy for a variety of different reasons/algorithms (DP, greedy, etc.). On the left, though, are all hard problems, which researchers have been unable to efficiently solve over many, many years. The fascinating thing about these hard problems is that *they are all difficult for the same reason*! This may seem incredibly counterintuitive—they're all different problems, and vary widely. How could they possibly all be the same difficulty?! Well, as we'll soon discuss, these are actually all the **same problem**! That is, these problems are all equivalent; as we'll soon discuss, these problems can all be *reduced* to each other.

P, NP

No, P/NP does not mean Pass/No Pass here. P and NP are what's known as **complexity classes**. You've probably heard of these before too; perhaps in the context of the Millennium Problems—each problem within this set of problems has a $1 million bounty, and one such problem is proving (or disproving!) that $\mathrm{P} \neq \mathrm{NP}$. But what do these *actually* mean?

Formally, NP denotes the class of *all search problems*. We formally defined these above, but, in short, a search problem must possess an efficient algorithm C to check if a solution S is valid for a given instance I. Efficient, here, means polynomial in $|I|$. Note that this includes both intractable (e.g. 3SAT) and tractable (e.g. 2SAT) problems.

P, meanwhile, denotes the class of all search problems that can be solved in *polynomial time*. Importantly, $\mathrm{P} \subseteq \mathrm{NP}$. (And thus, the Millennium problem essentially seeks to prove the

conjecture $\mathrm{P} \subset \mathrm{NP}$; generally speaking, we assume $\mathrm{P} \neq \mathrm{NP}$ is true. Proving the contrary would mean that all these "hard" problems are solvable in polynomial time!).

Reductions

Well, now that we've defined our complexity classes, and now that we've clarified our assumption that $\mathrm{P} \neq \mathrm{NP}$, how do we *prove* that these hard problems have no efficient algorithm? This is done via **reductions**, transformations that show one problem is *at least as hard* as another. Researchers have leveraged reductions to show that *all* the problems on the left side of the above table are essentially the exact same problem, as aforementioned. What's more, they have shown that these problems are in fact the *hardest* search problems in NP. If any of these problems has a polynomial time algorithm, then *every* problem in NP would have a polynomial time algorithm.

Let's first formalize the notion of a reduction. A *reduction* from search problem A to search problem B is a pair of polynomial time algorithms (f, h) such that

1. f transforms any instance I of A into an instance $f(I)$ of B
2. h transforms any solution S of $f(I)$ for B into a solution $h(S)$ of I for A. Or, if $f(I)$ has no solution for B, then it can be concluded that I has no solution for A.

Note how this reduction from $A \to B$ effectively shows that B is *at least as hard* as A, since, if there exists a polynomial time algorithm for B, there naturally exists a polynomial time algorithm for A derived from B's via the reduction.

> ⚠ **Make sure you understand that a reduction from $A \to B$ means that $B \geq A$, with regards to tractability/difficulty. This is easy to accidentally mix up!**

Now, we can formally define the class of the hardest search problems, too (the left side of the above table). A search problem is NP-Complete if *all* other search problems reduce to it.

> **Composition of Reductions**
> One final note about reductions is that they compose, i.e. $A \to B$ and $B \to C$ implies $A \to C$. This should hopefully make sense intuitively.

NP-Hard?

You might have heard of the term NP-Hard to refer to the hardest problems in computer science. So what's this NP-Complete term then?

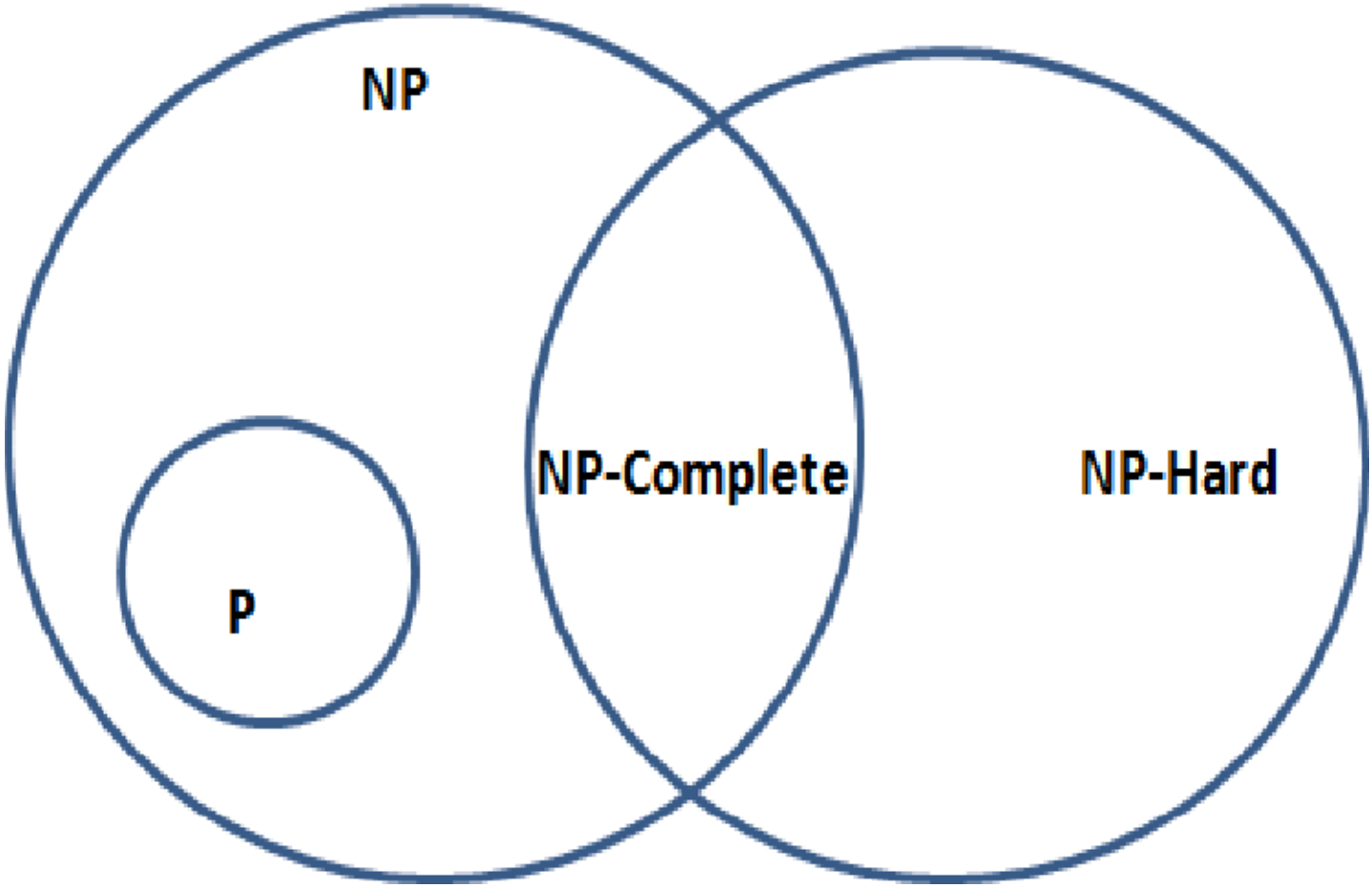
Well, NP-Hard is actually yet another complexity class. NP-Hard denotes the class of all problems at least as hard as NP-Complete problems.

Wait—I thought the problems in NP-Complete were the hardest problems??

Not quite. NP-Complete denotes the class of the hardest *search* problems. There exist non-search problems at least as hard as those in NP-Complete—those are the ones in NP-Hard.

Note that, similar to how $\mathrm{P} \subseteq \mathrm{NP}$, NP-Complete $\subseteq$ NP-Hard. Also, NP-Hard is not a subset of NP! Remember that NP denotes the class of all *search* problems. Thus, NP-Complete denotes the *intersection* of NP-Hard and NP.

Finally, you should hopefully be able to understand the following diagram!



Factoring

Factoring is a famously difficult search problem: given an integer n, the desired solution is the prime factorization of n. So, does Shor's Algorithm, i.e. a polynomial time quantum algorithm to prime factorize a number, solve all of NP?

Not quite. In fact, factoring isn't one of the hardest problems in NP, i.e. it isn't an NP-Complete problem! One critical difference between factoring and the NP-Complete problems is that, for factoring, there is no "or else" clause—there is *always* a prime factorization for n. In any case, factoring is within NP, but not NP-Complete. So, while quantum computers are able to efficiently solve factoring, it still remains to be seen whether or not all of NP can be solved efficiently by quantum computers! (Though, current research seems to indicate that it is very likely that this is not the case).

8.3 Reductions, Reductions, Reductions

Rudrata (s, t)-Path $\rightarrow$ Rudrata Cycle

Consider adding an additional vertex x and two new edges $x \rightarrow s$ and $t \rightarrow x$ to the graph G to produce G'. Then, any Rudrata cycle of the graph G must contain the edges $x \rightarrow s$ and $t \rightarrow x$. Proving this reduction's validity requires showing that it works for (1) when the Rudrata Cycle problem has a solution and (2) when it does not. Moreover, it must be shown that (3) the preprocessing and postprocessing functions take polynomial time.

1. Existing Solution

Then, the Rudrata (s, t)-Path problem must also have a solution. Consider that removing the added edges from the Rudrata cycle of G' produces the Rudrata path from $s \rightarrow t$ in G, as desired, since the other half of the cycle, $t \rightarrow s$, must traverse through x (otherwise x would not be visited by the cycle, and thus it would not be a valid Rudrata cycle for G').

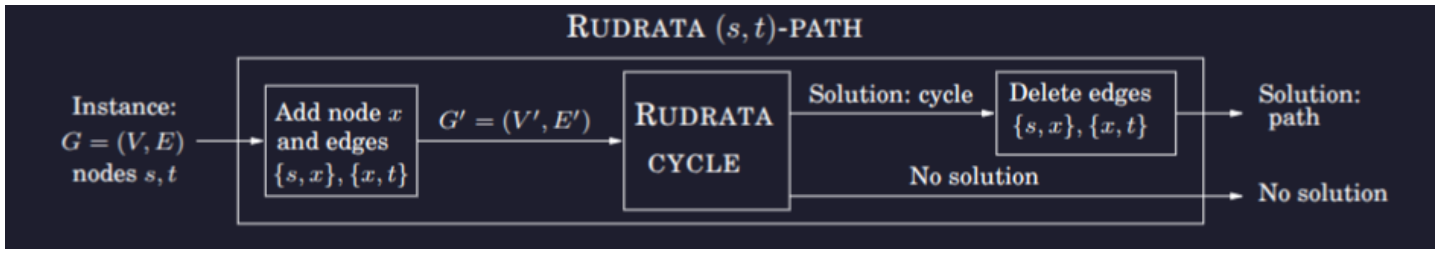
2. No Solution

It's typically easier to show the **contrapositive**, for this case. For this instance, it requires proving that, if there exists a Rudrata (s, t)-path in G, then there also exists a Rudrata cycle in G'. This is trivially true; just add the edges to the Rudrata path to close the cycle.

3. Preprocessing & postprocessing

The preprocessing step just requires adding one node and two edges. The postprocessing step just requires removing two edges. Thus, these steps are clearly polynomial time.

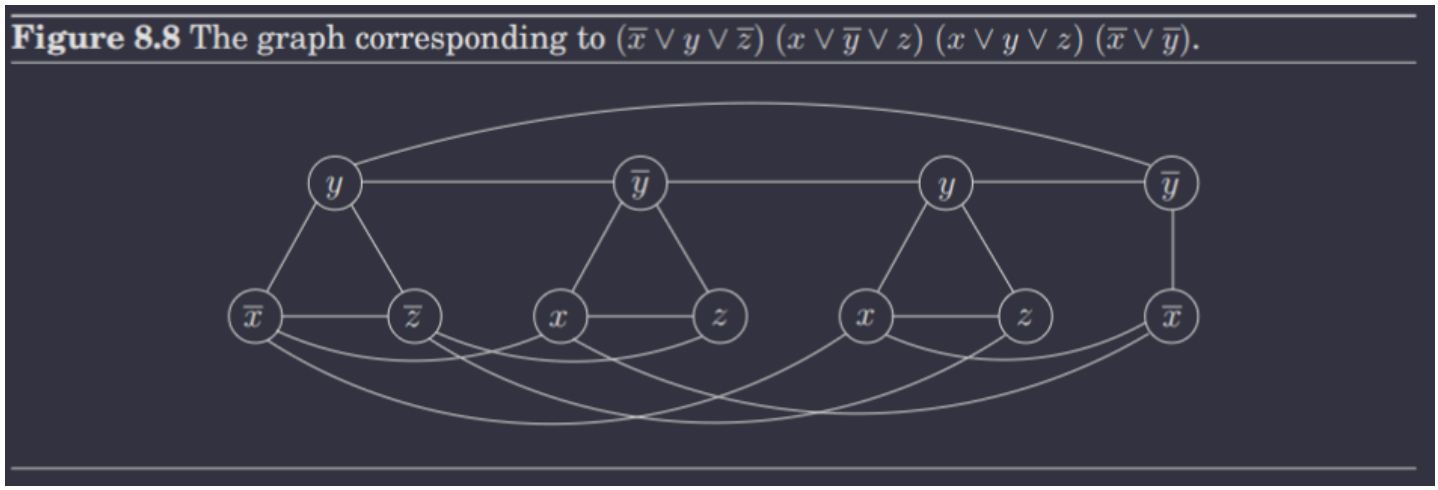
Thus, solving the Rudrata Cycle problem is *at least* as hard as the Rudrata (s, t)-Path problem. In other words, the Rudrata (s, t)-Path problem *reduces* to the Rudrata Cycle. Below is a diagram that illustrates the reduction visually.



> It's also possible to reduce in the other way, i.e. Rudrata Cycle $\rightarrow$ Rudrata (s,t)-Path. In other words, this pair of reductions show that these two problems are essentially the same problem.

3SAT $\rightarrow$ Independent Set

This reduction is most succinctly described by the following diagram.



In essence, the idea is that precisely one literal is chosen to be true in every clause, in order to make that clause true. Moreover, each node, WLOG x, is connected directly to *all* of its negations, i.e. all nodes $\overline{x}$. This ensures that, if x is chosen in a clause, $\overline{x}$ is never chosen in any other clause, since this would violate the property of an independent set. Note that multiple literals in a clause can actually be true, despite the graph ensuring only one is chosen for the independent set! This is okay, since we don't need to represent all literals that are true within a clause; only one needs to be true (in the independent set) to make that clause true. (Note that including one x node in the independent set, i.e. marking it as true, will not immediately add all other x nodes to the independent set; rather, it will simply preclude all $\overline{x}$ nodes from being part of the independent set).

SAT $\rightarrow$ 3SAT

This is an example of an interesting yet common reduction: reducing a problem to a *special case* of itself. In essence, this shows that the hardness of the problem is equivalent to the hardness of a special case.

The entire reduction relies on a single trick. Consider any clause in the SAT problem with more than three literals, i.e.

$$(a_1 \vee a_2 \vee \cdots \vee a_k) \qquad (1)$$

where $k > 3$. Then, we can rewrite this as a set of 3SAT clauses

$$(a_1 \vee a_2 \vee y_1)(\overline{y_1} \vee a_3 \vee y_2)(\overline{y_2} \vee a_4 \vee y_3) \dots (\overline{y_{k-3}} \vee a_{k-1} \vee a_k) \qquad (2)$$

The conversion is clearly polynomial in k (and thus polynomial in the SAT instance I's size, $|I|$), and the conversion from 3SAT back to SAT is clearly $O(1)$ (since all variables were already solved for in 3SAT).

> ⚠ **Make sure you see why these two expressions are equivalent! More formally, (1) is satisfied $\iff$ (2) is satisfied.**

We can go even *further* than this, though. We can additionally reduce the 3SAT problem to a variant such that *no variable appears in more than three clauses*. Consider any variable x that appears in $k > 3$ clauses. Then, replace its ith appearance with x_i, and add the clause

$$(\overline{x_1} \vee x_2)(\overline{x_2} \vee x_3) \dots (\overline{x_k} \vee x_1)$$

Note how x_i appears thrice, twice in the above expression and once in its original clause.

> ⚠ **Again, make sure you understand why the above expression ensures $x_1 = x_2 = \cdots = x_k$.**

Independent Set $\to$ Vertex Cover

We alluded to this reduction earlier in Section 8.1. Consider that a set of nodes S is a vertex cover of graph $G \iff V - S$ is an independent set of G.

Forward Direction:

We proceed via proof by contraposition. Let $V - S$ not be an independent set of G. Then, $\exists u, v \in V - S$ such that the edge $(u, v) \in E$. Then, $u, v \notin S$, and thus the edge (u, v) is not covered by S. Therefore, S is not a valid vertex cover.

Backward Direction:

Again, we can prove this by contraposition. The details are left as a (hopefully easy) exercise to the reader! >:)

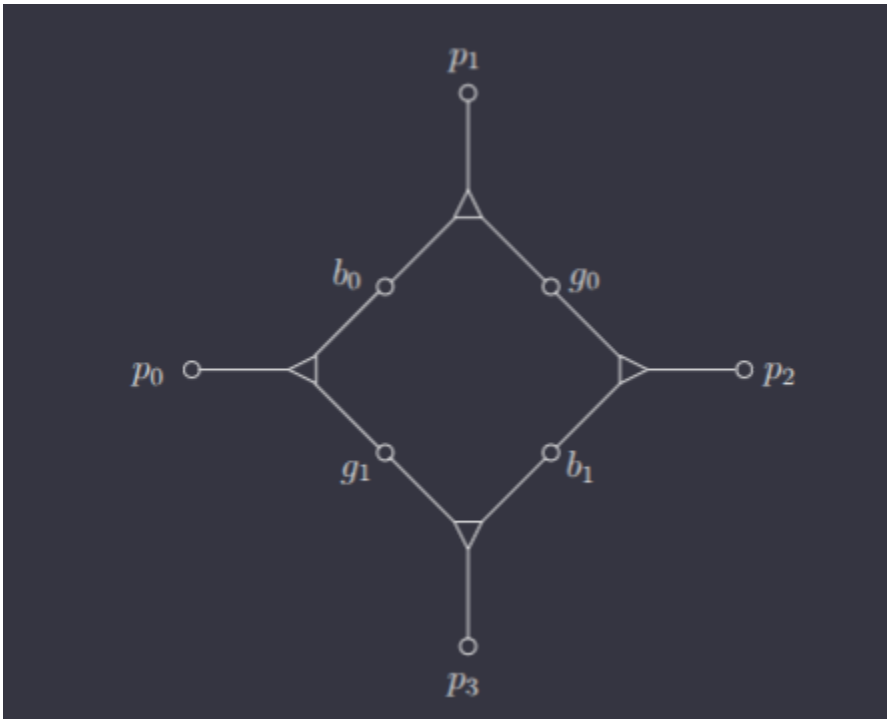
Independent Set $\to$ Clique

Define the *complement* of a graph $G = (V, E)$ to be $\overline{G} = (V, \overline{E})$, where $\overline{E}$ consists of all edges (u, v) not in E. Then a set of nodes S is an independent set of $G \iff S$ is a clique of $\overline{G}$. That is, the nodes in S have no edges between them in $G \iff$ all possible edges exist between them in $\overline{G}$. This should hopefully make sense intuitively.

3SAT $\rightarrow$ 3D Matching

This is a very unintuitive reduction. But, I'll do my best to explain the transformation.

Consider the following 3D Matching diagram, where each triangle node represents a triple.



Suppose b_0, b_1 and g_0, g_1 are not involved in any other triples. Some of the pets $p_0, \ldots, p_3$ must belong to other triples, of course, since at most two triples can be chosen here (and thus at most two pets can be "used up" here). In fact, precisely two pets must be "used up" here, since, in order for a perfect matching, b_0, b_1, g_0, g_1 must all be "used up" in this diagram, since they are not involved with any other triples.

Consider choosing the triple (b_0, g_0, p_1). Then, this forces the other triple in this diagram to be (b_1, g_1, p_3). Similarly, choosing the triple (b_0, g_1, p_0) forces the other triple to be (b_1, g_0, p_2). Therefore, any matching involving this *gadget* must include either both (b_0, g_0, p_1) and (b_1, g_1, p_3) or both (b_0, g_1, p_0) and (b_1, g_0, p_2). In other words, this gadget behaves like a *Boolean variable*!

So, to transform an instance of 3SAT into an instance of 3D Matching, we first create a gadget for *each* variable. Let the nodes for a variable x be denoted $b_{x0}, b_{x1}, g_{x0}, g_{x1}, p_{x0}, p_{x1}, p_{x2}, p_{x3}$. We

will let $x = \text{true}$ be denoted by the case where b_{x0} is matched with girl g_{x1}, and $x = \text{false}$ be denoted by the case where b_{x0} is matched with g_{x0}.

Now, consider a clause, e.g. $c = (x \vee \overline{y} \vee z)$. For each clause, we introduce a new boy b_c and new girl g_c. We will associate each with three triples, one for each literal in the clause. In essence, we want this boy and girl pair to be matched with a single pet in one of the literals' gadgets, such that this match implies that this literal was chosen to be true.

For this clause, we can have $(1)\ x = \text{true},\ (2)\ y = \text{false},\ (3)\ z = \text{true}$. For (1), we form the triple (b_c, g_c, p_{x1}). This is because, if x is chosen to be true, then p_{x0}, p_{x2} would be taken in the gadget. Then, the triple (b_c, g_c, p_{x1}) can be included in the 3D Matching, effectively marking that $x = \text{true}$ makes clause c true. In contrast, if x is chosen to be false, then p_{x1} would be taken in the gadget $\implies$ x is not used to make the clause true. We extend this to the other two literals, i.e. y has the triple (b_c, g_c, p_{y0}) and z has the triple (b_c, g_c, p_{z1}).

Additionally, we have to ensure that, for every occurrence of a literal in clause c, there is a different pet to match with b_c and g_c. For instance, if a literal is used in 5 different clauses, we won't have enough pets to match with b_c, g_c for each clause c! However, recall that we actually showed a reduction from 3SAT to a special case in which each *literal* appears in at most *two* clauses. Therefore, if we first reduce 3SAT to a special case, we can ensure each literal appears at most twice, in which case we actually do have enough pets—if a variable $x = \text{true}$, we have two pets p_1 and p_3 that can each be matched in clauses, and if $x = \text{false}$, we have p_0 and p_2.

⚠ **Note that a pet is only assigned if the corresponding literal is true in that clause. If the literal is $\overline{x}$, a pet is assigned to this literal if $x = \text{false}$. Conversely, if the literal is x, a pet is assigned if $x = \text{true}$.**

? **But I thought the reduction showed that each literal appeared in at most *three*, not two, clauses?** ›

I was confused about this too. Actually, the earlier reduction showed that each *variable* appeared in at most three clauses. Notably, though, in the expression derived to ensure $x_1 = x_2 = \cdots = x_k$, each variable was used once as x_i and once as $\overline{x_i}$. Meanwhile, the original clause contains one of x_i or $\overline{x_i}$. Therefore, each *literal*, i.e. x_i and $\overline{x_i}$ are the literals, appears at most twice. Confusing, I know; I wish the book explained this.

We're almost done now. The last thing we need to ensure is that *no pet is left unmatched*. For instance, consider a variable that is used only in one clause—it'll be left with an unmatched pet, which isn't allowed in 3D Matching! More precisely, with n variables and m clauses in 3D

SAT, precisely $2n - m$ pets will be left unmatched. Thus, it suffices to simply add $2n - m$ boy-girl couples that can match with every pet, and have them take up the remaining pets!

> **Why are exactly $2n - m$ pets left unmatched? ›**
>
> Each variable's gadget will have two pets that are not matched within the gadget itself. Each clause will take precisely one pet. Thus, $2n - m$ pets left unmatched. Note that $2n - m > 0$ always since each variable will appear at most thrice, and there are at least two literals in every clause.

3D Matching $\rightarrow$ ZOE

For each triple, create a variable x_i, where $x_i = 1$ means the triple was chosen. Our vector $\mathbf{x} = \langle x_1, \ldots, x_n \rangle$. Then, for each boy/girl/pet, let the triples containing it are $x_{j_1}, x_{j_2}, \ldots, x_{j_k}$. Then, we can set a constraint

$$\sum_{i=1}^{k} x_{j_i} = 1$$

since each boy/girl/pet can be included in at most one triple. Then, the rest is just solving the corresponding ZOE problem,

$$\mathbf{Ax} = \mathbf{1}$$

where each column of $\mathbf{A}$ corresponds to a triple, and each row corresponds to a boy/girl/pet.

ZOE $\rightarrow$ Subset Sum

This is a reduction between two special cases of ILP, and is literally just based on the fact that $0 - 1$ vectors can essentially act as binary representations of a number!

For instance, consider the ZOE problem with matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Think of the columns as binary; then, effectively,

$$\mathbf{A} = \begin{bmatrix} 18 & 5 & 4 & 8 \end{bmatrix}$$

. Then,

$$\mathbf{Ax} = \mathbf{1}$$

reduces to the subset sum problem where we are searching for a subset of $\{18, 5, 4, 8\}$ that sums to $11111_2 = 31$.

Well, *not quite*. There's still one more consideration—carries from addition can result in a valid solution for Subset Sum but an invalid solution for ZOE. To fix this, we can slightly modify the transformation: instead of thinking of the column vectors as integers in base 2, we consider them as integers in base $n + 1$. Therefore, this ensures that carries never occur, and thus a solution to Subset Sum is always a solution to ZOE.

ZOE $\rightarrow$ ILP

As aforementioned, ZOE is a *special case* of ILP. Then, this effectively means that ZOE reduces to ILP, since being able to solve ILP in polynomial time trivially equates to being able to solve ZOE in polynomial time, since there isn't even a transformation required.

> ⓘ **This is a very useful way of establishing that a problem is NP-Complete, i.e. by noting it is a *generalization* of a known NP-Complete problem.**

ZOE $\rightarrow$ Rudrata Cycle

This transformation is complex and fairly long, so I will attempt to explain it concisely as best as I can.

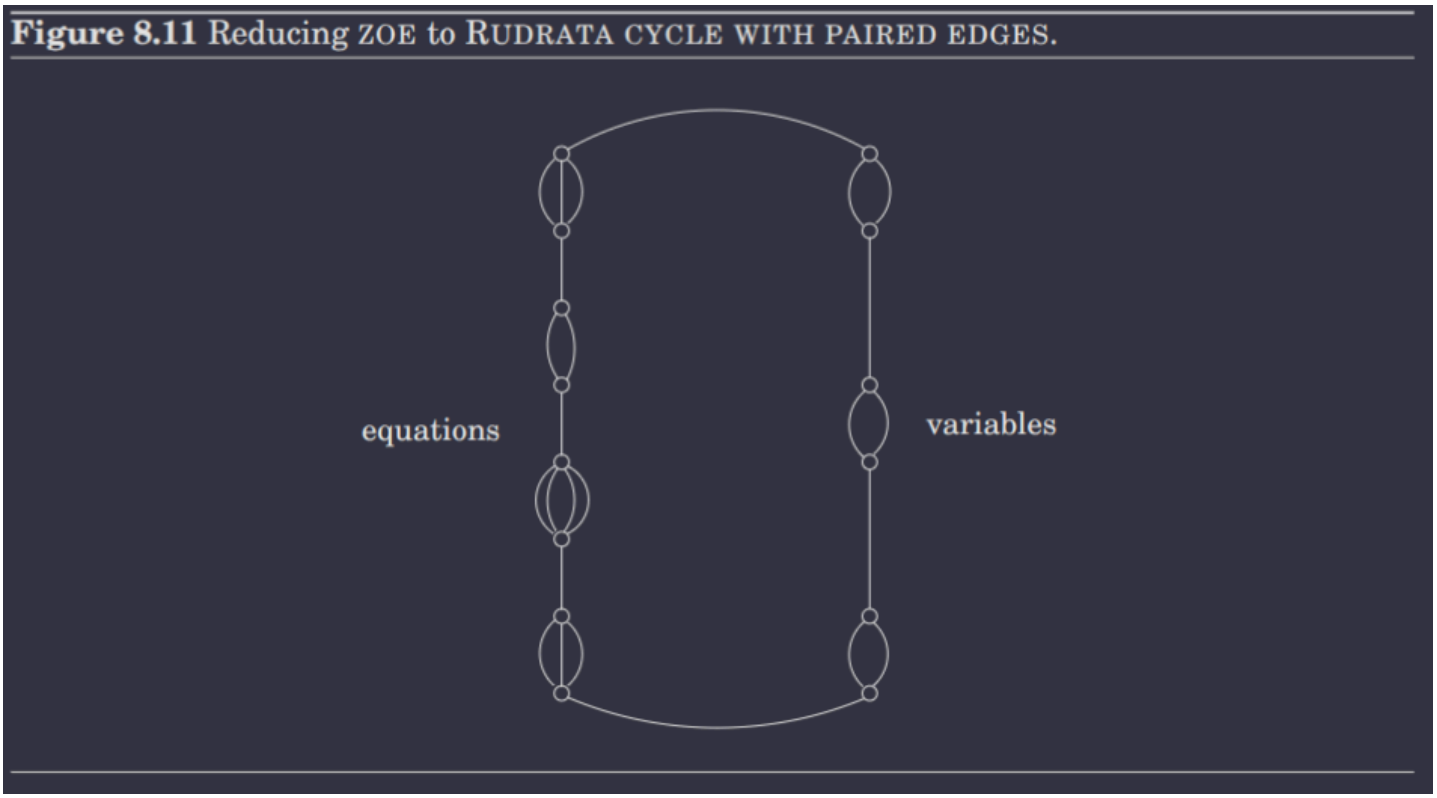
First, we will try to reduce ZOE to a generalization of Rudrata Cycle: *Rudrata Cycle with Paired Edges*, and then we will reduce this generalization to Rudrata Cycle. (Recall that this is valid since reductions are *transitive*).

ZOE $\rightarrow$ Rudrata Cycle with Paired Edges

In Rudrata Cycle with Paired Edges, a graph $G = (V, E)$ and a set $C \subseteq E \times E$ comprise the provided instance. The desired solution is a cycle that (1) visits all vertices once and (2) for every pair of edges $(e, e') \in C$, traverses *precisely one* of the two.

Now, consider the structure of a ZOE problem. We have n variables $x_1, \ldots, x_n$ that can either be 0 or 1, and m equations $x_{j_1} + \cdots + x_{j_k} = 1$, where $j_i \in \{1, \ldots, n\}$. For variable x_i, we create a component consisting of two nodes and two edges e_{i0} and e_{i1} between these two nodes, where the Rudrata cycle traversing e_{i0} corresponds to $x_i = 0$, and it traversing e_{i1} corresponds to $x_i = 1$. For equation i, we create a component consisting of two nodes and m edges

$e'_{i1}, \ldots, e'_{im}$, where e.g. the Rudrata cycle traversing e'_{i1} corresponds to the variable $x_{j_1} = 1$ in equation i. Then, we compose these components as shown in the diagram below.

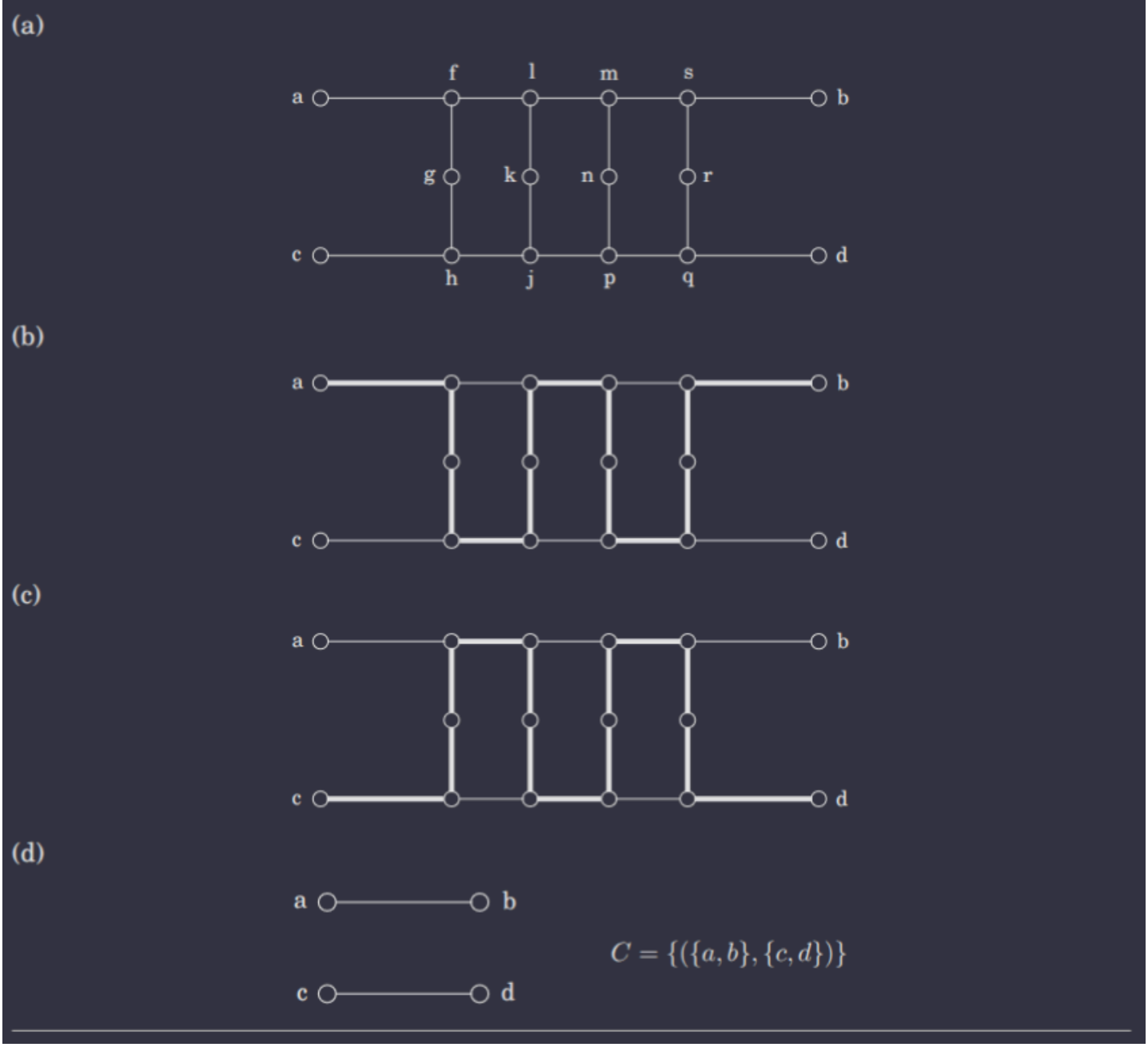


Then, for every equation, for every variable x_i appearing in it, we add to C the pair (e, e') where e corresponds to the edge representing x_i in the equation and e' corresponds to the edge represent $x_i = 0$. This ensures that if $x_i = 0$, it's never used as the value 1 in any equation. Therefore, ZOE reduces to the Rudrata Cycle with Paired Edges problem.

Rudrata Cycle with Paired Edges $\rightarrow$ Rudrata Cycle

This transformation can be summed up in a single diagram.

Figure 8.12 A gadget for enforcing paired behavior.



In essence, by replacing every pair of edges (e, e') with this gadget, we ensure that only one of $\{e, e'\}$ is traversed. Thus, the Rudrata Cycle with Paired Edges problem reduces to the Rudrata Cycle problem, and by the transitivity of reductions, ZOE reduces to the Rudrata Cycle Problem.

What if some edge e is involved in multiple pairs? >

Then we can simply concatenate all of its gadgets together. Make sure you see why this works!

Rudrata Cycle $\rightarrow$ TSP

Given a graph $G = (V, E)$, construct a TSP where the set of cities is precisely the same as V, the distance between cities u and v is 1 if $(u, v) \in E$ and otherwise is $1 + \alpha$, for some $\alpha > 1$. The budget b is $|V|$.

If G has a Rudrata cycle, then the same cycle is also a tour within the budget of the TSP instance, clearly. Conversely, if G has no Rudrata cycle, then there is no solution, as the cheapest possible TSP tour has a cost $\geq n + \alpha$ (since it traverses at least one edge that doesn't exist in G, and thus has length $1 + \alpha$).

Why the α parameter? Can't we just set $\alpha = 1$? ›

Varying α can lead to two interesting results. If $\alpha = 1$, then this TSP instance actually satisfies the triangle inequality, which produces a special case of TSP that, as we will discuss in Section 9, can be *efficiently approximated*.

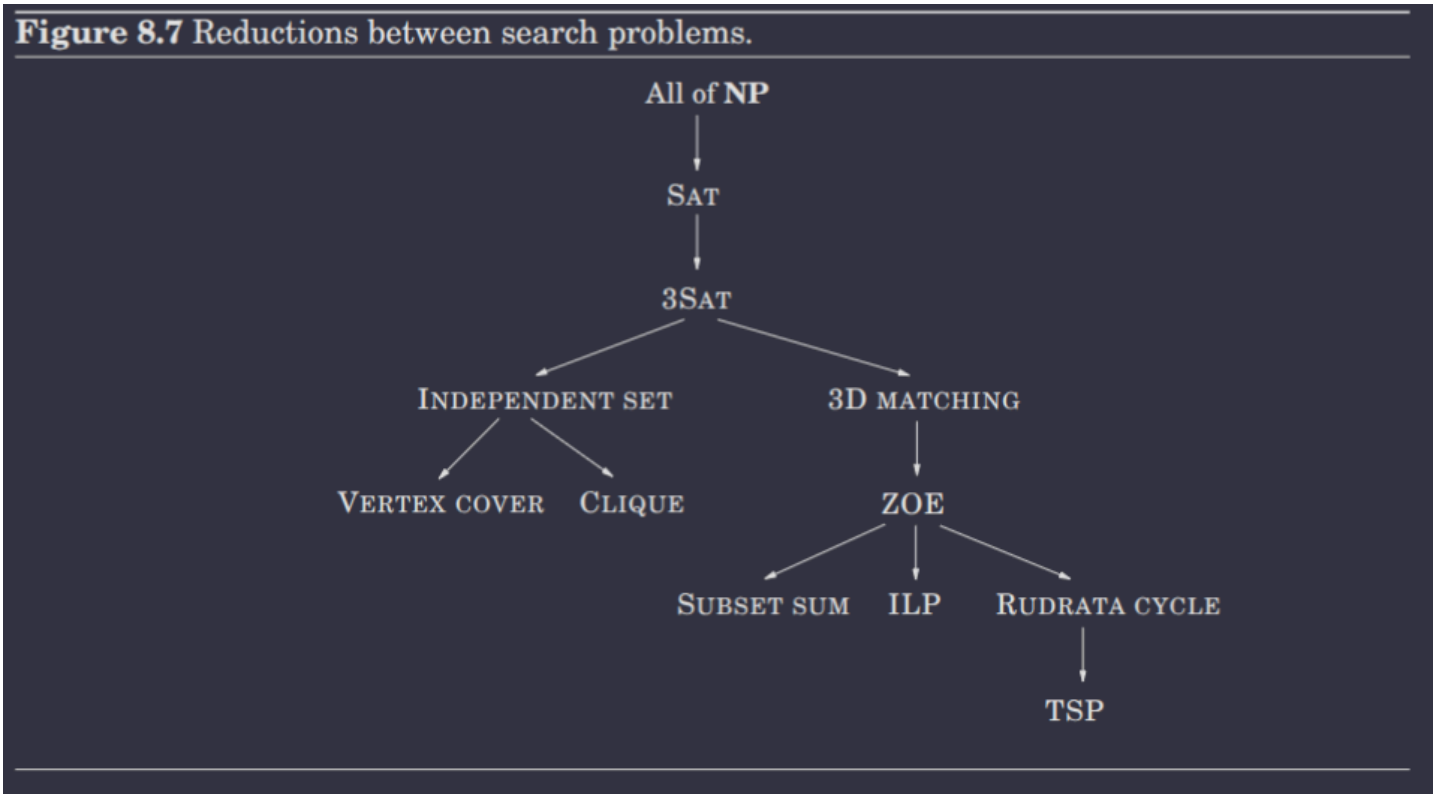
Conversely, if α is sufficiently large, then it has the property that it either (1) has a solution of cost $\leq n$ or (2) has a solution of cost $\geq n + \alpha$. Critically, there are 0 solutions with cost c such that $n < c < n + \alpha$. This is known as the **gap property**, and implies that unless $\mathrm{P} = \mathrm{NP}$, there exists no efficient approximation algorithm.

Any Problem in $\mathrm{NP} \rightarrow$ SAT

ts too hard... TL;DR is Any Problem in $\mathrm{NP} \rightarrow$ Circuit SAT $\rightarrow$ SAT.

Summary

Now, you should hopefully understand the tree of reductions shown in the below diagram :)



Aside: Unsolvable Problems

For all NP-Complete problems, there at least exists some algorithm to solve each, even if intractably so. For such problems, there is no existing algorithm at all! One such problem is the arithmetical version of SAT, where the provided instance is a polynomial equation in many variables and the desired solution is a satisfying assignment of the variables. Another very famous unsolvable problem is the halting problem.

9. Coping with NP-Completeness

> In this section, we discuss how to "cope" with NP-Complete problems. In other words, how to tackle NP-Complete problems you encounter in real life.

9.1 Intelligent Exhaustive Search

Backtracking

Backtracking is a technique that involves recursively exploring a search space, and then backpedaling steps that were taken. More importantly, though, backtracking actually takes advantage of the observation that it is often possible to *reject a solution by looking at just a small portion of it*.

Consider, for instance, a SAT problem with the clause $(x_1 \vee x_2)$. Then all assignments with $x_1 = x_2 = \text{false}$ can be eliminated without further exploration. Thus, here, we are able to quickly check and reject this *partial assignment*. So, how can we systematize this idea to effectively "backtrack"?

The critical idea is to *incrementally* grow a tree of partial solutions. Whenever we encounter a partial solution that doesn't work, we can immediately prune this subtree, and explore it no further. In other words, we *backtrack* out of this subtree where we know there are no valid solutions.

Formally, a backtracking algorithm requires a *test* function that receives a subproblem and quickly determines if

1. Failure: the subproblem has no solution
2. Success: a solution to the subproblem is found
3. Uncertainty: needs further exploration

For instance, for a SAT problem, if we recurse through the subproblem tree by, for each step/variable set, (1) removing all clauses that are already satisfied by setting this variable and (2) removing the variable from the remaining clauses that are not satisfied by setting this variable. Consider setting $x = 1$ in the SAT problem

$$(x \vee y)(\overline{x} \vee \overline{y} \vee z)(y \vee z)$$

Then, it would produce the subproblem

$$(\overline{y} \vee z)(y \vee z)$$

Therefore, for the SAT problem, the "test" function would declare failure if the subproblem has an empty clause, success if the subproblem has no clauses remaining, and uncertainty

otherwise.

Branch-and-Bound

We now generalize this principle of *pruning* the search space from search problems to optimization problems. WLOG, we will consider a generic minimization problem (a maximization problem will naturally follow a similar pattern).

The critical idea of branch-and-bound (for a minimization problem) is to quickly compute a *lower bound* on the cost function. This allows us to reject any subproblems whose lower bound is greater than the cost of an already-encountered solution (must be a full solution, not a subproblem!)—as this implies these subproblems must certainly not be optimal.

Consider an instance of TSP. We let a partial solution be a simple path $a \to b$ traveling through the set of vertices $S \subseteq V$, where $a, b \in S$. We denote the partial solution by $[a, S, b]$. For TSP, a will actually always be the start vertex, and the initial problem is $[a, \{a\}, a]$. Then, the corresponding subproblem is the best *completion of the tour*, i.e. the minimum cost complementary path $b \to a$ whose intermediates nodes are precisely $V - S$.

For each iteration of branch-and-bound, we extend a partial solution $[a, S, b]$ by a single edge (b, x) where $x \in V - S$. There are at most $|V - S|$ ways to do this, and each branch corresponds to a new subproblem $[a, S \cup \{x\}, x]$.

Now, the important part—how can we quickly compute a lower-bound on the completion, i.e. path $b \to a$, of the partial tour $[a, S, b]$. There are some very complicated solutions, but we will consider a very simple heuristic. The completion of the tour must consist of a path through $V - S$ plus edges $b \to \beta \in V - S$ and $\alpha \to a \in V - S$. Thus, its cost is at least the sum of the

- The lightest edge to a from $V - S$
- The lightest edge from b to $V - S$
- The minimum spanning tree of $V - S$

Make sure you understand why before moving on! But, in essence, since computing the minimum spanning tree of a graph has an efficient algorithm, this gives us our desired "quick" lower bound, which we can use to prune branches.

9.2 Approximation Algorithms

We now consider efficient algorithms that provide good approximations of the optimal solution to an NP-Complete problem. WLOG, we will consider minimization problems. First, a little bit of notation

- Let $\mathrm{OPT}(I)$ denote the value of the optimum solution. We will assume that $\mathrm{OPT}(I) \in \mathbb{Z}^+$ to make our math simpler
- Let $\mathcal{A}$ denote an arbitrary algorithm, typically an efficient one, and let $\mathcal{A}(I)$ be its produced solution to an instance I

Then, we say the **approximation ratio** of algorithm $\mathcal{A}$ is defined as

$$\alpha_{\mathcal{A}} = \max_{I} \frac{\mathcal{A}(I)}{\mathrm{OPT}(I)}$$

In other words, $1 \leq a_{\mathcal{A}} < \infty$ and it measures, multiplicatively, how much worse algorithm $\mathcal{A}$'s solution is for the *worst-case input*. That is, the worst possible ratio between $\mathcal{A}$'s solution and the actual optimal solution observed across all instances I.

Vertex Cover

In Section 5.4, we already observed that the Set Cover problem can be approximated within a factor of $O(\log n)$ by a greedy algorithm. Since the Vertex Cover problem is just a special case of the Set Cover, we already have an algorithm with approximation ratio $\alpha = O(\log n)$.

However, there exists a better approximation algorithm that is based on the idea of **matching**, i.e. a subset of edges that have no vertices in common. We call a matching *maximal* if no more edges can be added to it.

> ⚠ **A maximal matching is *not the same* as a maximum matching. A maximum matching is a matching with the most number of edges amongst all matchings. A maximal matching is simply a matching that cannot be expanded further without removing edges from it.**

Critically, *any* matching in a graph G provides a *lower bound* on OPT for the vertex cover problem. This is because each edge of the matching must have one of its endpoints in the set of any vertex cover, by definition!

Additionally, consider a set S that contains both endpoints of each edge in a maximal matching M. Then S must be a vertex cover; otherwise, it must not touch some edge $e \in E$, which would imply that M is not maximal since we could add e to it—a contradiction! (Since, if e is not covered by S, e has no vertices in common with any edge in M).

Note that this cover has precisely $2|M|$ vertices, i.e. two vertices for each edge. (And no less because no edges can share a common vertex). From before, we know that each edge of a matching must have, at minimum, one of its endpoints in the set of any vertex cover. Therefore, *any* vertex cover must have size at least $|M|$. Thus, the cover we found is at most

twice the size of the optimal vertex cover. Therefore, an algorithm that efficiently finds the maximal matching would have an approximation ratio of $\alpha \leq 2$.

It turns out that maximal matchings are exceedingly easy to generate—just repeatedly choose edges that are disjoint from those chosen already until no more can be added. Using a disjoint set union, this can be efficiently completed in $O(n \log n)$!

Clustering

The clustering problem asks us to divide a set of data points into groups of "close" points.

First, let's formalize the notion of distance. Let d denote the distance function, which may vary depending on the scenario. It must satisfy the usual *metric* properties

- $d(x, y) \geq 0, \ \forall x, y$
- $d(x, y) = 0 \iff x = y$
- $d(x, y) = d(y, x)$
- $d(x, y) \leq d(x, z) + d(z, y)$ (Triangle Inequality)

Then, the k-Cluster problem may be described as follows.

- Input:
  Points $X = \{x_1, \ldots, x_n\}$
  Distance metric $d(.,.)$
  An integer k
- Output:
  A partition of the points into k disjoint clusters $C_1, \ldots, C_k$ such that $C_1 \cup \cdots \cup C_k = X$.
- Goal:
  Minimize the maximum cluster diameter, i.e. minimize $\max_j \max_{x_a, x_b \in C_j} d(x_a, x_b)$.

Perhaps the easiest way to visualize this is covering a set of n points in 2D space with k circles of equal size, and trying to determine the smallest possible diameter of the circles.

This problem is NP-Hard, as you might've guessed. However, it has an exceedingly simple approximation algorithm!

The key idea is to pick k data points as *cluster centers* and then assign the remaining points to the center closest to it, where the cluster centers are chosen by iteratively choosing the next center to be as far as possible from the centers chosen so far (until k centers are chosen).

The algorithm clearly always returns a valid partition, and, in fact, the resulting diameter is guaranteed to be at most *twice* OPT.

Let $x \in X$ be the point farthest from $\mu_1, \ldots, \mu_k$ (i.e. the next center we would have chosen, if this was a $k+1$-Cluster problem). Let $r = \min_i d(x, \mu_i)$. Then every point in X must be within distance r of its cluster center. This is because the definition of x implies that $\nexists x' \in X$ such that $r' = \min_i d(x', \mu_i) > r$, i.e. there existed a point that was more than r units away from all cluster centers, and thus x was not, in fact, the farthest from $\mu_1, \ldots, \mu_k$, a clear contradiction. Thus, every point in X must be within distance r of its cluster center, and, by the *triangle inequality*, every cluster has diameter at most $2r$. Thus, this algorithm produces a solution with diameter at most $2r$.

But how does this solution's diameter relate to the optimal solution? Well, consider that we have identified a set of $k+1$ points $\{\mu_1, \mu_2, \ldots, \mu_k, x\}$ that have a pairwise distance of at least r. Then, any partition into k clusters must place two of these points within the same cluster—thus, the optimal partition into k clusters must have a diameter of at least r. In other words, the algorithm $\mathcal{A}$ presented above has an approximation ratio $\alpha_{\mathcal{A}} \leq 2$.

TSP

We can exploit the *triangle inequality* to produce an efficient approximation of variants of the Traveling Salesman Problem where the triangle inequality is satisfied! As with the previous algorithms, we consider an efficiently computable structure that is somewhat related to the best TSP tour—a **minimum spanning tree**.

Consider that removing any edge from a traveling salesman tour produces a path through all vertices, i.e. a spanning tree (albeit one without any branches). However, this does tell us that

$$\text{TSP cost} \geq \text{this path's cost} \geq \text{MST cost}$$

So, an MST already provides us with a nice lower bound on the TSP cost. However, remember that the goal of an approximation algorithm is to produce a solution that approximates the optimal TSP solution, not just bound the cost. How can we do this?

One thought is to use each edge of the MST twice, which can create a tour that visits all nodes at least once, but some nodes more than once. This tour has a length at most twice the MST cost, which therefore is at most twice the TSP cost, as evinced by the aforementioned inequality; however, this tour isn't quite a legal solution to the TSP, since it visits some nodes multiple times!

We can fix this by forcing the tour to simply *skip* any node it is about to revisit, and directly visit the next unvisited node in the list. By the triangle inequality, these bypasses can only

make the overall tour shorter; thus, this algorithm $\mathcal{A}$ has an approximation ratio $\alpha_{\mathcal{A}} \leq 2$.

General TSP

But what TSP variants which the triangle inequality does not apply to? Well, as it turns out, these variants provably* *cannot* be approximated.

*"Provably" as in there does not exist an efficient approximation provided $\mathrm{P} \neq \mathrm{NP}$.

Recall in Section 8.3 that we discussed a reduction from the Rudrata Cycle/Path problem to the TSP problem. In short, we can reduce any Rudrata Path problem on a graph G to an instance $I(G, \alpha)$, where $\alpha \in \mathbb{Z}^+$ and can be arbitrary, of TSP such that (1) if G has a Rudrata path, then $\mathrm{OPT}(I(G, \alpha)) = n$ and (2) if G has no Rudrata path, then $\mathrm{OPT}(I(G, \alpha)) \geq n + \alpha$. Interestingly enough, if *any* efficient, approximate solution for the general TSP existed, the Rudrata Path problem becomes efficiently solvable!

Let $\mathcal{A}$ be an efficient approximation algorithm for TSP with approximation ratio $\alpha_{\mathcal{A}}$. Consider an instance G of Rudrata Path. Create an instance $I(G, C)$ of TSP using the constant $C = n\alpha_{\mathcal{A}}$. Then, $\mathcal{A}$ will output one of two things on I. (1) It will output a tour of length at most $\alpha_{\mathcal{A}}\mathrm{OPT}(I(G, C)) = n\alpha_{\mathcal{A}}$. Or (2) it will output a tour of length at least $\mathrm{OPT}(I(G, C)) > n\alpha_{\mathcal{A}}$. If (1) occurs, then we can conclude that G has a Rudrata Path, and return the produced TSP as the Rudrata Path!

Therefore, if $\mathrm{P} \neq \mathrm{NP}$, there cannot exist a polynomial-time approximation algorithm for TSP since we know the Rudrata Path problem is NP-Hard. Note that this conclusion is drawn from the **gap property** hinted towards in Section 8.3.

Knapsack

Finally, we come to an approximation algorithm for a maximization problem—knapsack! Recall that the dynamic programming solution we saw to the knapsack problem in Section 6.4 had a runtime of $O(nW)$ or $O(nV)$, which slight modification, which both are, unfortunately, not polynomial in just n, since W and V can be exponential in n.

Consider the $O(nV)$ algorithm. What if, if V is extremely large, we just scale down each value? For instance, if

$$v_1 = 117,586,003 \qquad v_2 = 738,493,291 \qquad v_3 = 238,827,453$$

we could lose some precision but make the algorithm several orders of magnitude more efficient by using

$$v_1 = 117 \qquad v_2 = 738 \qquad v_3 = 238$$

.

More precisely, let $v_{\max} = \max_i v_i$. Then we may rescale each value v_i to $\hat{v}_i = \left\lfloor v_i \cdot \frac{n}{\epsilon v_{\max}} \right\rfloor$, and run the dynamic programming algorithm on this rescaled version of the problem. Note that $\epsilon > 0$ is some approximation factor specified in the input to the algorithm.

Since the rescaled values $\hat{v}_i$ are all at most $\frac{n}{\epsilon}$, the algorithm runs in $O(nV) = O\left(n\left(n \cdot \frac{n}{\epsilon}\right)\right) = O(n^3/\epsilon)$. So the algorithm itself is efficient. What about the approximation factor, though?

As you may have guessed, ϵ is related to the approximation factor of this algorithm. I'll spare the details, but it can be shown that, the approximation factor is $\alpha_{\mathcal{A}} \geq 1 - \epsilon$, i.e. the value K output by this algorithm is related to OPT by $K \geq \mathrm{OPT}(1 - \epsilon)$.

Approximability Hierarchy

All NP-Complete optimization problems can be classified into one of the following categories.

- No finite approximation ratio is possible (e.g. TSP)
- An approximation ratio is possible, but there are limits to how small it can be / it's directly proportional to the input size (e.g. Vertex Cover, k-Cluster, TSP w/ triangle inequality)
- An approximation ratio of $\approx \log n$ is possible (e.g. Set Cover)
- The approximation ratio has no limits, i.e. polynomial approximation algorithms with error ratios arbitrarily close to zero exist (e.g. knapsack)

⚠ **Remember that this all relies on $\mathrm{P} \neq \mathrm{NP}$.**

Also, note that these algorithms often perform much better than their worst-case approximation ratio.

9.3 Local Search Heuristics

Local search is a technique inspired by evolution—incrementally introducing random mutations and keeping those that improve performance. It can be applied to nay optimization problem!

The most naive version is iteratively replacing the current solution with a better one nearby, i.e. within its **neighborhood**. What exactly defines the "neighborhood" of a solution is the central design decision in local search.

TSP

We'll first consider the application of local search heuristics to TSP.

The most obvious choice of definition for "closeness" in TSP is if two tours differ in only a few edges. It's impossible for two tours to differ in only one edge, so we'll first consider two tours as "close" if they differ in exactly two edges. Then, let the *2-change* neighborhood of tour s be the set of tours that can be obtained by removing two edges of s and adding in two distinct edges.

So... how well does this do? That is, what's the runtime, and does it always return the most optimal solution?

Unfortunately, it's not too clear what the answer to this question is. Each iteration is fast, since a tour only has $O(n^2)$ neighbors, but it's unclear how many iterations will be needed, and whether or not the final tour is *globally optimal* or just *locally optimal*.

We can somewhat reduce the local optimality limitation by, say, extending the neighborhood to *3-change*; however, the size of a neighborhood increases to $O(n^3)$, thus increasing runtime. Moreover, extending to 3-change does not eliminate the problem entirely; globally suboptimal but locally optimal minima still exist, though fewer. Thus, there's a balance between the algorithm's runtime and its closeness to the global optimum; the right balance is typically found via iterative experimentation.

Graph Partitioning

Graph partitioning may be described as the following problem.

- Input:
  Undirected graph $G = (V, E)$ with nonnegative edge weights
  Real number $\alpha \in (0, 1/2]$
- Output: A partition of the vertices into two groups A and B, each of size at least $\alpha|V|$
- Goal: Minimize the capacity of the cut (A, B)

Note that we actually saw a special case of this in Section 8, *Balanced Cut*. In fact, we will consider an even stricter variant of Balanced Cut here for designing a local search heuristic: we will focus on the special case $\alpha = \frac{1}{2}$. (Interestingly enough, the general problem of Graph Partitioning reduces to this special case).

Now, we must define what a neighborhood is / our notion of closeness. Let (A, B) with $|A| = |B|$ be a candidate solution. Then, we will define its neighborhood to be the set of all solutions that can be obtained by swapping one pair of vertices across the cut, i.e.

$$\{(A - \{a\} + \{b\}, B - \{b\} + \{a\}) \mid (a, b) \in A \times B\}$$

As with TSP, though, many globally suboptimal but locally optimal solutions exist. How can we fix this?

Dealing with Local Optima

Randomization and Restarts

Randomization adds a bit of variance to our local search! It's typically applied in two ways: (1) picking a random initial solution and (2) randomly choosing a local move when several are optimal.

Randomization is best used by running local search repeatedly with a different random seed on each invocation. If the probability of reaching a good local optimum on any given run is p, then it's expected that within $O(1/p)$ runs, such a solution will be found.

Simulated Annealing

Randomization is great, but it's not always effective. For some problems, as the size grows, the ratio of bad to good local optima increases, often exponentially. **Simulated annealing** is a different approach that allows moves within the neighborhood that actually *increase the cost*—the idea is that locally suboptimal moves might actually bring the search out of a bad local optima and closer to a good local optima/the global optimum. Simulated annealing introduces a new parameter: a **temperature** T, where a higher temperature increases the likelihood that locally suboptimal solutions are chosen for each move.

Typically, simulated annealing will start off with a large T and slowly reduce it to zero (at which point it is effectively the same as normal local search). This allows the local search to initially wander around freely, and gradually begin to prefer the more optimal solutions, allowing it to converge on a solution.

Simulated annealing is a great strategy; however, it's also a much more expensive technique, as higher temperatures inevitably result in more local moves until convergence. Thus, it frequently requires significant experimentation to choose the *annealing schedule*, i.e. the schedule for slowly reducing the temperature. Ultimately, though, the solution quality improvements that simulated annealing provides are often worth it!

Fun Fact >

Simulated annealing was inspired by the physics of crystallization. Search up "annealing" if you want to know more :)

10. Quantum Computing

> Note: these notes are fairly short and do not comprehensively discuss the background behind quantum computing. This is because I had prior knowledge regarding quantum computing, and wasn't particularly interested in writing a whole essay here about it since I already knew... sorry :(
>
> That said, I do recommend books like Quantum Mechanics: The Theoretical Minimum (Susskind and Friedman) or Quantum Computation and Quantum Information (Nielsen and Chung) for learning the basics (and perhaps even more!)

Quantum Complexity

BQP is the class of all problems efficiently solvable by a quantum computer. It is believed that $\mathrm{P} \subseteq \mathrm{BQP}$, but $\mathrm{BQP} \nsubseteq \mathrm{NP}$ and $\mathrm{NP} \nsubseteq \mathrm{BQP}$. In other words, there are problems within BQP that are not efficiently verifiable (e.g. how do you verify a physics simulation?) and there are problems within NP that are *probably* not efficiently solvable by a quantum computer.

Elitzur-Vaidman Bomb

Consider a box that may or may not contain a bomb, that interacts as follows with a qubit:

- If it is empty, and we send $|x\rangle = a|0\rangle + b|1\rangle$, nothing happens and $|x\rangle$ is returned.
- If it has a bomb, and we send $|x\rangle$ as before, the bomb measures $|x\rangle$ in the $\{|0\rangle, |1\rangle\}$ basis. If the outcome is $|0\rangle$, $|0\rangle$ is returned. Otherwise, the bomb explodes.

We want to determine a way to distinguish if the box has a bomb or not without causing an explosion if the box has a bomb.

In the classical world, there exists no such algorithm for this. Sending $|0\rangle$ does not confirm anything, and sending $|1\rangle$ will cause the bomb to explode if it exists. However, in the quantum world, we can create an opportunity to determine if a bomb exists without causing an explosion!

Let $|x\rangle = |0\rangle$ initially. Then, apply a $\frac{\pi}{4}$ (45°) rotation to $|x\rangle$ to produce $|x\rangle = |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. Then, we send $|x\rangle$ through the box.

- If there is a bomb, it has an explosion probability of 50%.
- Alternatively, if there is no bomb, $|x\rangle$ will be returned.

Finally, we rotate the output of the box by $\frac{\pi}{4}$ again and measure $|x\rangle$.

- If $|0\rangle$ is observed, we output "bomb."

- If $|1\rangle$ is observed, we output "not sure."

Let's walk through the possible outcomes to see why this is the case.

- If there is no bomb, $|+\rangle$ will be output by the box, which a $\frac{\pi}{4}$ degree rotation will then bring to $|1\rangle$.
- If there is a bomb, but no explosion, $|0\rangle$ will be output by the box, which a $\frac{\pi}{4}$ degree rotation will then bring to $|+\rangle$. Measuring $|+\rangle$ has a $\frac{1}{2}$ chance of measuring $|0\rangle$ and a $\frac{1}{2}$ chance of measuring $|1\rangle$.
- If there is a bomb, but an explosion, it doesn't matter—we know there's a bomb, but we haven't accomplished our goal.

So, if there isn't a bomb, we'll always receive "not sure." After running this test n times, there is a $1 - \frac{1}{2^n}$ probability that the box has a bomb.

If there is a bomb, there is a 50% chance that the bomb's measurement results in $|1\rangle$, an explosion. Otherwise, there is an addition 50/50 between returning "bomb" and returning "not sure." Thus, there is a $\frac{1}{2}$ chance of an explosion, a $\frac{1}{4}$ chance of returning "bomb," and a $\frac{1}{4}$ chance of returning "not sure."

However, $\frac{1}{2}$ is still a high chance—we can do better!

As before, let $|x\rangle = |0\rangle$ initially. However, our rotation angle now changes to $\theta = \left(\frac{\pi}{2}\right)/N$, where N is some chosen parameter. Then, for $t = 1, \ldots, N$, we perform the same procedure.

1. Rotate $|x\rangle$ by θ.
2. Send $|x\rangle$ through the box. Let $|x\rangle$ be the value returned by the box.

Then, at the end, measure $|x\rangle$. If $|0\rangle$, return "bomb." Otherwise, return "empty."

> Note that, in this procedure, the $|x\rangle$ sent into the box changes between rounds.

Let's analyze the possibilities again.

Empty:
The box will output $|x\rangle$ each time. After N iterations, $|x\rangle$ will have rotated the full $N \cdot \theta = \frac{\pi}{2}$ radians. Thus, $|x\rangle = |1\rangle$ at the end.

Bomb:
Each time $|x\rangle$ passes through the box, if the bomb doesn't explode, $|0\rangle$ is returned. If there is never an explosion, and $|x\rangle = |0\rangle$ at the end, we know there is a bomb in the box. (Since empty

implies $|x\rangle$ should be $|1\rangle$). But then, what's the probability of the bomb exploding? Well, the probability of the bomb exploding at $t = i$ is $\sin^2\theta$. The probability of the bomb exploding at some $t \in \{1, \dots, N\}$ is

$$P[\text{explosion}] = P[\text{explosion at } t = 1] + \cdots + P[\text{explosion at } t = N]$$

since the events are all disjoint. Thus,

$$P[\text{explosion}] = N\sin^2\theta$$

For small θ, $\sin\theta \leq \theta$, so

$$P[\text{explosion}] \leq N\theta^2 = N\left(\frac{\pi/2}{N}\right)^2 \leq \frac{2.5}{N}$$

Thus, for this algorithm, if the box is empty, it will output "empty" with a probability of 1. If there is a bomb, the box will explode with probability $\frac{2.5}{N}$, and otherwise output "bomb." The time complexity, meanwhile, is $O(N)$.

> N is known as the safety level—if avoiding an explosion is extremely important (especially if it's an actual bomb...), increasing N will decrease the explosion probability, but increase the time complexity.

11. Randomized Algorithms

Types of Random Algorithms

Las Vegas: An algorithm that will always produce a correct result, but whose runtime will vary.
Monte Carlo: An algorithm that may or may not produce a correct result, but whose runtime is constant.

Primality Testing

Fermat Test

A naive solution would be to simply factor a number into its prime factors to determine if it's prime. However, the fastest algorithm for this, the *General Number Field Sieve*, runs in $O(C^{n^{1/3}\log(n)^{2/3}})$, i.e. exponential time. Can we do better?

Recall **Fermat's Little Theorem**:

> If N is prime, then $a^{N-1} \equiv 1 \pmod{N}$, $\forall a \in \{1, \ldots, N-1\}$.

We can thus develop a very simple test known as the **Fermat Test** for testing if a number is prime.

1. Pick $a \in \{1, \ldots, N-1\}$ uniformly at random
2. If $a^{N-1} \equiv 1 \pmod{N}$, output prime. Otherwise, output composite.

However, primality testing is not as simple as just choosing an a and checking if $a^{N-1} \equiv 1 \pmod{N}$. If N is composite, this congruence could also be true, for certain values of a. In essence, the Fermat Test guarantees that, if N is prime, it will output prime, but if N is composite, it may output prime or composite.

Thus, the Fermat Test will not always be correct. In fact, you may notice that this seems to fit the definition of a Monte Carlo randomized algorithm! Let's examine this further, though. In particular, we will determine an *upper bound* on the probability of some a "passing" the Fermat Test (i.e. outputting prime) when N is composite, for all N.

ⓘ Fermat Witness

Any a that fails the Fermat Test when N is composite, i.e. outputs the correct result / $a^{N-1} \not\equiv 1 \pmod{N}$, is known as a **Fermat witness**. If a is coprime to N, we further describe

is as a **nontrivial** Fermat witness, since if a is not coprime to N, it is trivially a Fermat witness. (See the proof below for an explanation).

> ⚠ **Carmichael Numbers**
>
> Note that there do exist composite numbers that pass the Fermat Test for all coprime a, i.e. for all nontrivial Fermat witnesses. For simplicity, we will ignore these.

Claim:
Let N be composite, and $S = \{1, \ldots, N-1\}$. Suppose there exists at least one $a \in S$ such that a is a nontrivial Fermat witness (i.e. N is not a Carmichael number). Let F be the set of Fermat witnesses. Then $|F| \geq \frac{1}{2}|S|$.

Proof:
We partition S into four disjoint subsets:

$$\begin{aligned} A &= \{a \in S \mid a^{N-1} \equiv 1 \pmod{N},\ \gcd(a, N) = 1\} \\ B &= \{b \in S \mid b^{N-1} \not\equiv 1 \pmod{N},\ \gcd(b, N) = 1\} \\ C &= \{c \in S \mid c^{N-1} \not\equiv 1 \pmod{N},\ \gcd(c, N) \neq 1\} \\ D &= \{d \in S \mid d^{N-1} \equiv 1 \pmod{N},\ \gcd(d, N) \neq 1\} \end{aligned}$$

Consider D. As aforementioned, any number that is not coprime to N is trivially a Fermat witness. Why? Let $k = \gcd(d, N) \neq 1$. If $d^{N-1} \equiv 1 \pmod{N}$, then $\exists q \in \mathbb{Z}$ such that $d^{N-1} - 1 = qN$. However, $k \mid d^{N-1}$ and $k \mid N \implies k \mid qN$, which implies $k \mid 1$, which is contradictory. Thus, $D = \emptyset$.

Meanwhile, B represents the nontrivial Fermat witnesses, C represents the trivial Fermat witnesses, and A represents the non Fermat witnesses. Note that $F = B \cup C$. Also note that $A, B, C \neq \emptyset$ since $1 \in A$, B is nonempty as given by the claim, and C is nonempty since N is composite.

Assume for contradiction that $|F| = |B \cup C| < \frac{1}{2}|S|$. Then $|A| > \frac{1}{2}|S|$. Let $A = \{a_1, \ldots, a_k\}$, where $k > \frac{1}{2}|S|$. Choose an arbitrary $b \in B$, and consider Ab, i.e.

$$Ab = \{a_1 b \pmod{N},\ \ldots,\ a_k b \pmod{N}\}$$

Ab has two useful properties that we will prove.

1. $Ab \subseteq B$.
2. The elements of Ab are distinct, i.e. $|Ab| = |A|$.

(1) $(a_i b)^{N-1} \equiv a_i^{N-1} b^{N-1} \equiv b^{N-1} \not\equiv 1 \pmod{N}$. Moreover, $\gcd(a_i b, N) = 1$ since $\gcd(a_i, N) = 1$ and $\gcd(b, N) = 1$. Note that $a_i b \pmod{N}$ is also clearly in S, due to modular arithmetic. Therefore, $Ab \subseteq B$.

(2) Suppose for contradiction that $a_i b \pmod{N} \equiv a_j b \pmod{N}$ for $i \neq j$. This implies that $(a_i - a_j)b \equiv 0 \pmod{N}$, i.e. $N \mid (a_i - a_j)b$. Since $\gcd(b, N) = 1$, we must have $N \mid (a_i - a_j)$. However, $a_i, a_j \in S$ implies that $a_i, a_j < N$, i.e. $a_i - a_j = 0 \implies a_i = a_j \implies i = j$. This yields a contradiction, and thus the elements of Ab must be distinct.

The above two properties thus imply that, if $B \neq \emptyset$, then there are at least k elements in B, where $k > \frac{1}{2}|S|$. This produces a contradiction, however, and therefore it must be true that $|F| \geq \frac{1}{2}|S|$.

□

Proof Source

Therefore, given an integer N with unknown primality, if we run the Fermat Test for k rounds and no Fermat witness is identified, then either n is one of the (very rare) Carmichael numbers or n is prime with probability $\geq 1 - \left(\frac{1}{2}\right)^k$. Thus, we have a nice, randomized Monte Carlo test for primality! (Effective for all but the Carmichael numbers).

Miller-Rabin Test

The Miller-Rabin test is a more robust test than the Fermat Test, in the sense that it works on *all* numbers, including Carmichael numbers. It is a Monte Carlo algorithm, like the Fermat test, and always outputs "prime" if n is prime and outputs "composite" if n is composite with probability $> \frac{3}{4}$. Thus, it returns the correct answer with probability $> 1 - (\frac{1}{4})^k$ in k rounds.

The Miller-Rabin test relies on a different witness of compositeness, based on the following theorem.

> If p is prime, then all integer roots of $x^2 \equiv 1 \pmod{p}$ satisfy $x \equiv 1 \pmod{p}$ or $x \equiv -1 \pmod{p}$.

Proof:

$$\begin{aligned} x^2 \equiv 1 \pmod{p} &\implies p \mid (x^2 - 1) \\ &\implies p \mid (x-1)(x+1) \end{aligned}$$

Since p is prime, either $p \mid x - 1$, $p \mid x + 1$, or both. This means $x - 1 \equiv 0 \pmod{p}$, $x + 1 \equiv 0 \pmod{p}$, or both. Thus, the only possible roots are $x = \pm 1$. Finally, we note that these are not extraneous, i.e. $(\pm 1)^2 \equiv 1 \pmod{p}$ is in fact true.

□

> **Corollary:** Let $n, x \in \mathbb{Z}$ such that $x^2 \equiv 1 \pmod{n}$. If $x \not\equiv \pm 1 \pmod{n}$, then n must be composite.

Now, we can introduce some similar notation as for the Fermat Test. For a composite number n, we denote x as a **root witness** of n's compositeness if $x^2 \equiv 1 \pmod{n}$ and $x \not\equiv \pm 1 \pmod{n}$. By definition, a root witness is a *nontrivial root*.

We'll subsequently derive the intuition and logic behind the Miller-Rabin test.

In essence, the Miller-Rabin test determines that n is composite by using both Fermat and root witnesses. Let us assume $n > 2$ and is odd (otherwise it is immediately composite), and let a be chosen uniformly at random from $S = \{1, \ldots, n-1\}$.

Since n is odd, $n-1$ is even. Let $n - 1 = 2^r \cdot d$, where $d \equiv 1 \pmod{2}$. We can substitute this into the Fermat Test's equation to produce

$$a^{n-1} \equiv a^{2^r \cdot d} \pmod{n}$$

Suppose that $a^{2^r \cdot d} \equiv 1 \pmod{n}$, i.e. a is not a Fermat witness. Consider rewriting the expression as

$$(a^{2^{r-1} \cdot d})^2 \equiv 1 \pmod{n}$$

Then, if

$$a^{2^{r-1} \cdot d} \not\equiv \{1, -1\} \pmod{n}$$

We have successfully identified a root witness of compositeness, and thus n is correctly identified as composite. However, there's still more to this!

If $a^{2^{r-1} \cdot d} \equiv -1 \pmod{n}$, then we can do nothing further. However, if $a^{2^{r-1} \cdot d} \equiv 1 \pmod{n}$, we can form yet *another* equation of the form $x^2 \equiv 1 \pmod{n}$. In particular,

$$(a^{2^{r-2} \cdot d})^2 \equiv 1 \pmod{n}$$

Thus, as long as we get $a^{2^{r-\alpha} \cdot d} \equiv 1 \pmod{n}$ and $r - \alpha > 0$, we can continue trying to find a root witness.

Therefore, we can now define the steps of the Miller-Rabin algorithm. For simplicity, we consider only a single round. (Note: any output immediately returns from the function).

1. Express $n - 1 = 2^r \cdot d$ for some odd d

2. Choose $a \in S = \{1, \ldots, n-1\}$ uniformly at random
3. If $a^{2^r \cdot d} \not\equiv 1 \pmod{n}$, output "composite (Fermat)"
4. For $y = r-1$ to $y = 0$,
    1. If $a^{2^y \cdot d} \not\equiv \{1, -1\} \pmod{n}$, output "composite (Root)"
    2. If $a^{2^y \cdot d} \equiv -1 \pmod{n}$, output "probably prime"
5. output "probably prime"

Actually, though, this algorithm is a little more computationally expensive than necessary, since it must compute $a^{2^r \cdot d}$. We can instead start from a^d and square this quantity r times in the for loop. Thus, a more optimized version looks like this.

1. Express $n - 1 = 2^r \cdot d$ for some odd d
2. Choose $a \in S = \{1, \ldots, n-1\}$ uniformly at random
3. Let $y = 0$
    1. If $a^{2^y \cdot d} \equiv 1 \pmod{n}$, output "probably prime"
    Why? All future squares will be 1, so there are no root witnesses.
    2. If $a^{2^y \cdot d} \equiv -1 \pmod{n}$, output "probably prime"
    Why? Same reason as 3.1.
4. For $y = 1$ to $y = r-1$
    1. If $a^{2^y \cdot d} \equiv 1 \pmod{n}$, output "composite (Root)"
    2. If $a^{2^y \cdot d} \equiv -1 \pmod{n}$, output "probably prime"
    Why? Same reason as 3.1.
5. Output "composite"

> **Why stop at $y = r-1$?**
>
> Suppose we continued to $y = r$. Then it must have been the case that $a^{2^{r-1} \cdot d} \not\equiv \{1, -1\} \pmod{n}$. Now, consider $a^{2^r \cdot d}$. If it is $1 \pmod{n}$, we have a root witness. Otherwise, if it is $\not\equiv 1 \pmod{n}$, we have a Fermat witness (recall $a^{2^r \cdot d} = a^{n-1}$).

Finally, it can be shown that the single-round Miller-Rabin primality test will output composite for a composite n for a randomly chosen a with probability $> \frac{3}{4}$. The proof of this is nontrivial, and thus is excluded from these notes. In conclusion, though, after running the Miller-Rabin test for k rounds on a composite n, it will find a witness of compositeness with probability $> 1 - \left(\frac{1}{4}\right)^k$.

Agrawal-Kayal-Saxena (AKS)

This is a *deterministic* algorithm for primality test in $O(n^{12})$ (actually, it's been shortened to $O(n^6)$ now). This is effectively a derandomized version of the Miller-Rabin algorithm. For brevity, this algorithm's details are left out.

Minimum Cut

We previously saw an algorithm in Section 7 that solved for the maximum flow / minimum cut for a pair of vertices s, t deterministically in $O(nm^2)$, or $O(n^5)$ for dense graphs. However, we now consider a related, but slightly different problem—given an unweighted, undirected graph $G = (V, E)$, we seek the minimum cut across *all* possible pairs s, t, i.e. the cut with the globally minimum number of crossing edges. There actually exists a Monte Carlo algorithm for this problem that runs in $O(n^2)$: **Karger's Algorithm**.

Karger's Algorithm proceeds simply as follows.

1. For $i = 1, \ldots, n-2$
    1. Pick a uniformly random edge e_i
    2. *Contract* e_i, i.e. merge its vertices (u, v) into a "*supervertex*"

At the end of the loop, we are left with two supervertices, and we return the value of the cut between these two supervertices, which effectively represent the two sets of vertices we've divided the graph into.

...what? How does this work??

The critical idea behind this seemingly nonsensically-simple algorithm is that contracting is much less likely to contract the edges crossing the minimum cut. Why? Because the minimum cut, by definition, has the least number of crossing edges of any cut. Thus, it is by far more likely that an edge that does not cross the minimum cut is chosen, considering that there are many edges in the graph and the minimum cut edges form only a minuscule subset.

> ⓘ **Astute readers may notice that the idea of this algorithm is very similar to the randomized algorithm we briefly discussed in** Section 5. **:)** ›
>
> In fact, that algorithm is better!

Let's not stop here though—we'd like to formalize this intuition, and derive the probability that Karger's algorithm does, in fact, output the minimum cut. In particular, we will prove the following theorem.

> **Theorem:** Let $C = (S, \overline{S})$ be a minimum cut of size k.
> $\Pr[\text{Karger's Algorithm Result} = C] =\geq \frac{1}{\binom{n}{2}} = \frac{2}{n(n-1)}$.

Proof:
First, let's outline some definitions. Let G_i be the graph at the beginning of the ith iteration of the algorithm, with base case $G_1 = G$, and let H_i be a "happy" (^w^) event, i.e. the event in which the ith contracted edge e_i doesn't cross the cut C. Let A denote the event that $\text{Karger's Algorithm Result} = C$.

Then,

$$\begin{aligned}
\Pr[A] &= \Pr[H_1 \wedge H_2 \wedge \cdots \wedge H_{n-2}] \\
&= \Pr[H_1] \cdot \Pr[H_2 \mid H_1] \cdot \Pr[H_3 \mid H_1, H_2] \dots \Pr[H_{n-2} \mid H_1, \dots, H_{n-3}] \\
&\geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdots \frac{1}{3} \\
&= \frac{2}{n(n-1)}
\end{aligned}$$

The inequality follows from the following

$$\begin{aligned}
\Pr[H_i \mid H_1, \dots, H_{i-1}] &= 1 - \Pr[\text{Contract edge in } C \mid H_1, \dots, H_{i-1}] \\
&= 1 - \frac{\#\text{ crossing edges of } C}{\#\text{ edges in } G_i} \\
&= 1 - \frac{k}{\frac{1}{2}k(n-i+1)} = 1 - \frac{2}{n-i+1} = \frac{n-i-1}{n-i+1}
\end{aligned}$$

Where the number of edges in G_i follows from the following sequence of facts

1. $\text{MinCut}(G_i) \geq k$ (since any cut in G_i corresponds precisely to some cut in G)
2. Number of vertices in G_i, $|G_i|$, is $n - (i-1) = n - i + 1$, since $i - 1$ vertices have been removed
3. Degree of each vertex in $G_i \geq k$ (otherwise, if $\deg v < k$, minimum cut is less than k with $S = v$)
4. Number of edges in G_i is $\frac{1}{2}\sum_{v \in G_i} \deg v \geq \frac{1}{2}\sum_{v \in G_i} k = \frac{1}{2}k|G_i| \geq \frac{1}{2}k(n-i+1)$.

Thus, the probability of success, $\Pr[A]$, is $\geq \frac{1}{\binom{n}{2}} \approx \frac{2}{n^2}$. In fact, this scales linearly with the number of valid minimum cuts, i.e. the number of cuts C with the minimum number of crossing edges. So the probability of returning a minimum cut with c minimum cuts in the graph is $\frac{c}{\binom{n}{2}}$. In general, though this is not known a priori—thus, the lower bound on the probability of success is still $\frac{2}{n^2}$. Therefore, it suffices to run this algorithm $O(n^2)$ times, and take the minimum cut seen across all runs. For instance, repeating $20n^2$ times ensures that the probability a minimum

cut is not seen in any run is $\left(1-\frac{2}{n^2}\right)^{20n^2} < 0.01$ [Source]. So, the total runtime for Karger's Algorithm (to derive a minimum cut with high probability) is effectively $O(n^4)$.

12. Online Algorithms

> An **online algorithm** is one that can process data one-by-one and make more informed decisions based on past data. Essentially, an adaptive learning algorithm.

Experts

Consider a set of N experts that each make a prediction about a certain event for T time steps. Let $\mathcal{U}$ be the set of possible predictions. The problem proceeds as follows.

1. At time step t, each expert makes a prediction; let $\mathcal{E}^t \in \mathcal{U}^N$ be the vector of predictions.
2. The algorithm makes a prediction a^t, and then the actual outcome o^t is revealed.

The goal is to minimize the number of mistakes, i.e. minimize $|M|$, where $M = \{t \mid a^t \neq o^t\}$.

Perfect Expert

Let us first consider the situation in which there exists a perfect expert, i.e. an expert that makes zero mistakes.

> **Claim:** There exists an algorithm that $\lceil \log_2 N \rceil$ mistakes.

Proof.
At each time step t, the algorithm consider all experts who have yet to make a mistake, and predict the majority prediction amongst their predictions. Note that, for every mistake made, the set of experts that have yet to make a mistake reduces in size by *at least* a factor of 2. Since there exists a perfect expert, this algorithm can make at most $\lceil \log_2 N \rceil$ mistakes before it the set of experts who have yet to make a mistake consists solely of the perfect expert; at which point no more mistakes will be made. Thus, at most $\lceil \log_2 N \rceil$ mistakes will be made by this algorithm.

□

Bounded Mistakes

Now, we generalize the situation a bit. There no longer exists a perfect expert, but there exists an expert that makes at most m^* mistakes.

> **Claim:** There exists an algorithm that makes at most $M \leq m^*(\lceil \log_2 N \rceil + 1) + \lceil \log_2 N \rceil$.

We can essentially just extend the previous algorithm. We essentially divide the time duration T into epochs: each epoch, we run the algorithm from the perfect expert scenario; once the set of experts is empty, we start a new epoch with all N experts.

In each expert, each expert makes at least one mistake. Thus, the number of completed epochs is at most m^*. In each completed epoch, we make at most $\lceil \log_2 N \rceil + 1$ mistakes, and in the last epoch (which does not complete, since at least one expert will no longer make mistakes), at most $\lceil \log_2 N \rceil$ (to determine the now-perfect expert). Thus, the algorithm makes at most $m^*(\lceil \log_2 N \rceil + 1) + \lceil \log_2 N \rceil$ mistakes.

□

Weighted Majority Algorithm

But, we can do better—with an algorithm known as the *weighted majority algorithm*.

First, we describe the structure of the algorithm. The intuition behind the algorithm is to bias our predictions towards experts that appear to perform better. We assign a weight w_i to each expert $i \in \{1, \dots, N\}$. We denote the weight of an expert i at the beginning of round t as $w_i^{(t)}$. Initially, all weights are 1. Also, let $e_i^{(t)}$ denote the prediction of expert i in round t.

1. In round t, predict the outcome that maximizes the sum of the weights of experts that predicted it. Formally, $a^t = \arg\max_{u \in \mathcal{U}} \sum_{i:\, e_i^{(t)} = u} w_i^{(t)}$.
2. Then, after the outcome o^t is revealed, we update the weights for the next round. If $e_i^{(t)} = o^t$, $w_i^{(t+1)} = w_i^{(t)}$. Otherwise, if the expert was wrong, $w_i^{(t+1)} = w_i^{(t)} \cdot (1 - \epsilon)$, where ϵ is a parameter chosen earlier. We explain the choice of ϵ in more detail later.

$\epsilon = 1/2$

> **Claim:** For this algorithm, if $\epsilon = \frac{1}{2}$, the number of mistakes made by the weighted majority algorithm (WM) is at most
>
> $$2.41(m^* + \log_2 N)$$
>
> This is also commonly expressed as
>
> $$2.41m^* + O(\log_2 N)$$

Proof.
Let Z^t represent the aggregate weight of the set of experts at time step t. That is, $Z^t = \sum_i w_i^{(t)}$.

Note that

- $Z^1 = N$, since $w_i^{(t)} = 1, \forall i$ initially.
- $Z^{t+1} \leq Z^t, \forall t$, since experts' weights are only maintained or decreased.

Also, consider the event where this algorithm makes a mistake in round t. Then, the sum of weights of the wrong experts is higher than the sum of the weights of the correct experts. That is,

$$\sum_{i:\, e_i^{(t)} \neq o_t} w_i^{(t)} \geq \sum_{i:\, e_i^{(t)} = o_t} w_i^{(t)}$$

Considering that

$$Z^t = \sum_{i:\, e_i^{(t)} \neq o_t} w_i^{(t)} + \sum_{i:\, e_i^{(t)} = o_t} w_i^{(t)}$$

We can derive

$$Z^t \leq 2 \sum_{i:\, e_i^{(t)} \neq o_t} w_i^{(t)}$$

Now, consider the sum of the weights of the next round, Z^{t+1}.

$$\begin{aligned} Z^{t+1} &= \sum_{i:\, e_i^{(t)} \neq o_t} w_i^{(t+1)} + \sum_{i:\, e_i^{(t)} = o_t} w_i^{(t+1)} \\ &= \frac{1}{2} \sum_{i:\, e_i^{(t)} \neq o_t} w_i^{(t)} + \sum_{i:\, e_i^{(t)} = o_t} w_i^{(t)} \\ &= Z^t - \frac{1}{2} \sum_{i:\, e_i^{(t)} \neq o_t} w_i^{(t)} \\ &\leq \frac{3}{4} Z^t \end{aligned}$$

Where the last step follows from the inequality we previously derived.

In other words, the sum of all the weights decreases by at least a factor of 25% if the algorithm makes a mistake. Now consider any expert i, such that it has made m_i mistakes after t rounds. Let the algorithm have made M mistakes. Then,

$$\left(\frac{1}{2}\right)^{m_i} = w_i^{(t+1)} \leq Z^{t+1} \leq Z^t \left(\frac{3}{4}\right)^M = N\left(\frac{3}{4}\right)^M$$

Rearranging gives

$$M \leq \frac{m_i + \log_2 N}{\log_2\left(\frac{4}{3}\right)} \leq 2.41(m_i + \log_2 N)$$

Since we are guaranteed an expert makes at most m^* mistakes, we note that

$$M \leq 2.41(m^* + \log_2 N)$$

as desired.

□

Arbitrary ϵ

Now, we generalize our claim and proof to arbitrary* ϵ. As in, $\epsilon \in (0,\ 1/2)$.

> **Claim:** For the weighted majority algorithm, if $\epsilon \in (0,\ 1/2)$, we guarantee $M \leq 2(1+\epsilon)m^* + O\left(\frac{\log N}{\epsilon}\right)$.

Proof.
We leave out most of the analytical details for brevity, due to similarities with the previous proof. In short, we derive

$$Z^{t+1} \leq \left(1 - \frac{\epsilon}{2}\right) Z^t$$

Subsequently, we derive

$$(1-\epsilon)^{m_i} \leq Z^t \leq Z^1 \left(1 - \frac{\epsilon}{2}\right)^M = N\left(1 - \frac{\epsilon}{2}\right)^M \leq N \exp(-\epsilon M/2)$$

Rearranging gives

$$M \leq \frac{-m_i \log(1-\epsilon) + \ln N}{\epsilon/2}$$

Then, using the inequality

$$-\ln(1-\epsilon) = \epsilon + \frac{\epsilon^2}{2} + \frac{\epsilon^3}{3} + \cdots \leq \epsilon + \epsilon^2,\ \epsilon \in [0, 1]$$

gives us

$$M \leq 2\frac{m_i(\epsilon + \epsilon^2)}{\epsilon} + O\left(\frac{\log N}{\epsilon}\right)$$

Which simplifies nicely to

$$M \leq 2(1+\epsilon)m_i + O\left(\frac{\log N}{\epsilon}\right)$$

Given that one expert will make at most m^* mistakes, this naturally gives us our tightest upper bound.

□

Approximation Bound

Note that the above weighted majority algorithm can have a mistakes bound as close as possible to $2m^*$ as desired (by making ϵ smaller). In fact, this is as close as we can get; at least, with a deterministic algorithm.

> **Claim:** No deterministic algorithm $\mathcal{A}$ can do better than a factor of 2, compared to the best expert.

Consider a scenario with two experts E_1 and E_2. Let the set of choices be $\mathcal{U} = \{0, 1\}$. Let E_1 always predict 0 and E_2 always predict 1. Given that $\mathcal{A}$ is deterministic, an adversarial set of outcomes may be prepared such that $\mathcal{A}$'s predictions are wrong. For instance, if $\mathcal{A}$ is the weighted majority algorithm, if we always tiebreak by choosing E_1, a sequence of outcomes $1, 0, 1, 0, 1, 0 \ldots$ will result in $\mathcal{A}$ never being right. Regardless, for all deterministic algorithms $\mathcal{A}$, at least one of E_1 and E_2 will have an error rate of $\leq \frac{1}{2}$. (More precisely, their error rates should add to 1, since E_1 predicts 0 always and E_2 always predicts 1; therefore, at least one has an error rate $\leq \frac{1}{2}$). Yet, due to the adversarially chosen outcomes, $\mathcal{A}$'s error rate is always 1. Therefore, $\mathcal{A}$ will make, at best, twice as many mistakes as the best expert. Thus, it's not possible to design a deterministic algorithm that, for all possible scenarios of experts and outcomes, makes less than twice as many mistakes as the best expert.

□

Randomized Weighted Majority

Of course, the above proof does not apply to randomized algorithms. So, can we do better?

The *randomized weighted majority* algorithm (RWM) was designed precisely for this reason. In short, the weights evolve in the exact same way; however, the prediction at each time step is randomly chosen according to the distribution of the weights of the experts. You may imagine this as randomly choosing a single expert's opinion, weighted by the distribution. Succinctly,

$$\mathrm{P}[a^t = u] = \frac{\sum_{i:\, e_i^{(t)} = u} w_i^{(t)}}{Z^t}$$

> **Claim:** Let $\epsilon = \frac{1}{2}$. For any fixed sequence of predictions, the expected number of mistakes made by the randomized weighted majority algorithm is at most
>
> $$\mathrm{E}[M] \leq (1 + \epsilon) m^* + O\left(\frac{\log N}{\epsilon}\right)$$

Regret

The gap $\epsilon m^* + O\left(\frac{\log N}{\epsilon}\right)$ between the algorithm's expected performance and that of the best expert is known as the *regret* of the algorithm. We will see soon that this algorithm has effectively negligible regret.

Proof.

Let F^t be the fraction of total weight assigned to incorrect experts at time t, i.e.

$$F^t = \frac{\sum_{i:\, e_i^{(t)} \neq o^t} w_i^{(t)}}{Z^t}$$

Also, note that $\mathrm{E}[M] = \sum_{t \in [T]} F_t$, i.e. the expected number of mistakes made by the algorithm is the sum of the fractions of weight for the incorrect experts at each time step t. This derives itself from linearity of expectation; the expected number of mistakes at time step t is simply F_t, the probability of making a mistake at time step t. Thus, the expected number of mistakes in total is the sum of all such F_t.

By the weight adjustment procedures, we derive

$$Z^{t+1} = Z^t((1 - F^t) \cdot 1 + F^t \cdot (1 - \epsilon)) = Z^t(1 - \epsilon F^t)$$

Once again, consider an expert that has made m_i mistakes after t time steps.

$$(1-\epsilon)^{m_i} \leq Z^{t+1} = Z^1 \prod_{t'=1}^{t} (1 - \epsilon F_{t'}) \leq N e^{-\epsilon \sum F_t} = N e^{-\epsilon \mathrm{E}[M]}$$

Taking logarithms of both sides, we get

$$m_i \ln(1-\epsilon) \leq \ln N - \epsilon E[M]$$

And using the approximation $-\log(1-\epsilon) \leq \epsilon + \epsilon^2$ and rearranging gives

$$E[M] \leq m_i(1+\epsilon) + \frac{\ln N}{\epsilon}$$

And, of course, given that one expert makes at most m^* mistakes, we derive that this algorithm is expected to make at most $m^*(1+\epsilon) + \frac{\ln N}{\epsilon}$ mistakes.

□

ϵ

$\epsilon = \sqrt{\frac{\log N}{T}}$

$$\begin{aligned} E[M] &\leq (1+\epsilon)m^* + \frac{\log N}{\epsilon} \\ &\leq m^* + \epsilon m^* + \sqrt{T \log N} \\ &\leq m^* + \epsilon T + \sqrt{T \log N} \\ &\leq m^* + 2\sqrt{T \log N} \\ \frac{E[M]}{T} &\leq \frac{m^*}{T} + 2\sqrt{\frac{\log N}{T}} \end{aligned}$$

And since $\lim_{T \to \infty} 2\sqrt{\frac{\log N}{T}} = 0$, this algorithm, with this choice of $\epsilon = \sqrt{\frac{\log N}{T}}$, has effectively negligible regret for each time step.

Generalized Experts

Finally, we conclude with the generalization of the expert's problem, and a similar "negligible regret" algorithm that achieves $E[\text{Total Regret}] \leq O(\sqrt{T \log N})$.

In the general setting, there exists a set $\mathcal{A}$ of N actions you can take at each step (i.e., like N experts). On day t, one action $i \in \mathcal{A}$ must be chosen; let this action be denoted i^t. Afterwards, the cost $c_i^{(t)} \in [0, 1]$ is revealed (i.e. like 1 means a mistaken prediction, 0 means correct; only, in this generalized problem, the cost can be $0.5, 0.123$, etc.). The goal is to minimize the cost function. Or, more specifically, the goal is to minimize the regret—the difference between the cost of the most optimal action (i.e., total cost of choosing this action across all days) and the actual cost.

$$\sum_{t=1}^{T} c_{i^t}^{(t)} = \min_{a \in \mathcal{A}} \left\{ \sum_{t=1}^{T} c_a^{(t)} \right\} + \text{Regret}$$

Multiplicative Weights

The **Multiplicative Weights** algorithm is similar to the Randomized Weighted Majority algorithm. We initialize weights similarly, i.e. $w_i^{(1)} = 1$. For each time step t,

1. Let $Z^t = \sum_i w_i^{(t)}$.
2. Select action $i \in \mathcal{A}$ with probability $w_i^{(t)} / Z^t$.
3. Costs $\{c_1^{(t)}, \ldots, c_N^{(t)}\}$ are then revealed.
4. Set $w_i^{(t+1)} = w_i^{(t)} \cdot (1 - \epsilon c_i^{(t)})$.

We won't prove this (mainly because the lecture notes don't have a proof...), but we can show that selecting $\epsilon = \sqrt{\frac{\log N}{T}}$, like before, results in $E[\text{Regret}] \leq O(\sqrt{T \log N})$, or $E[\text{Regret}]/T \leq O(\sqrt{(\log N)/T})$. (In fact, it results in the same upper bound on the number of mistakes made).

Multiplicative Weights: Applications

The generalization of the experts problem, and the corresponding "negligible regret" multiplicative weights algorithm, is powerful and can be applied to many different problems. We briefly discuss a few of these applications.

Minimax Theorem (Zero Sum Games)

Consider a zero sum game with the payoff matrix

$$M = \begin{bmatrix} M_{11} & \dots & M_{1n} \\ \vdots & \ddots & \vdots \\ M_{n_1} & \dots & M_{nn} \end{bmatrix}$$

Where $-1 \leq M_{ij} \leq 1$.

Provided mixed strategies $p = \begin{bmatrix} p_1 \\ \vdots \\ p_n \end{bmatrix}$ (row) and $q = \begin{bmatrix} q_1 \\ \vdots \\ q_n \end{bmatrix}$ (column), we calculate

$$\text{Score}(p, q) = \sum_{i,j} p_i M_{ij} q_j = p^T M q$$

The **Minimax Theorem** states that

$$\min_q \max_p p^T M q = \max_p \min_q p^T M q$$

In other words, the score of the game is the same, regardless of whether the column player plays first (LHS) or the row player plays first (RHS).

> We briefly introduce some notation. Let $v_1' = \max_p \min_q p^T M q$ be known as the *gain-floor*—the minimum payoff the row player will receive given the column player's attempt to minimize their payoff. Let $v_2' = \min_q \max_p p^T M q$ be known as the *loss-ceiling*—the maximum loss the column player will experience given the row player's attempt to maximize their payoff.

Proof.

It's easy to show the $\geq$ direction. In natural language, this direction implies that moving first is, at the very least, not an advantage (i.e. the situation of the column player playing first will not produce a higher score than the situation of the row player playing first). This should make sense, intuitively, but we will formalize this intuition.

Consider that, for any pair of strategies (p', q'), we have

$$\min_q (p')^T M q \leq (p')^T M q' \leq \max_p p^T M q'$$

Hopefully this makes sense, intuitively. The LHS is choosing a q vector such that it minimizes the score, based on the value of p'. This is clearly $\leq$ the score of the pair of strategies (p', q'), since q is chosen to be score-minimizing. Meanwhile, the score of (p', q') is $\leq$ the RHS, since the RHS is choosing a score-maximizing p vector, based on the value of q'.

From the RHS of the inequality, we note that

$$(p')^T M q' \leq \max_p p^T M q'$$
$$\min_q (p')^T M q \leq \min_q \max_p p^T M q$$

This inequality holds for all p', since the previous inequality held for all pairs (p', q'). Therefore, it clearly holds if p is the score-maximizing value, i.e.

$$\max_p \min_q p^T M q \leq \min_q \max_p p^T M q$$

However, it is comparatively difficult to show the $\leq$ direction. To do so, we must either use the strong duality of LPs, or use the multiplicative weights algorithm.

Intuitively, we can use online learning to allow these players to repeatedly play the game and converge on the optimal solution. Consider the following procedure. For $t = 1, \ldots, T$ time steps,

1. The column player selects picks a strategy $q^{(t)}$ using multiplicative weights.
2. The row player chooses the "best response" strategy $p^{(t)} = \arg\max_{p^{(t)}} \{(p^{(t)})^T M q^{(t)}\}$.
3. The cost vector $c^{(t)} = p^{(t)} \cdot M$ for the column player is then revealed.
4. The column player's expected loss is then $\ell^{(t)} = (p^{(t)})^T M q^{(t)}$.

Let us define $\overline{p} = \frac{1}{T} \sum_{t=1}^{T} p^{(t)}$ and $\overline{q} = \frac{1}{T} \sum_{t=1}^{T} q^{(t)}$, i.e. the averages of the chosen strategies over all time steps. We aim to show that $\overline{p}$ and $\overline{q}$ are "approximate minimax strategies." In other words, we aim to show that

$$\max_p p^T M\overline{q} \leq \min_q \overline{p}^T Mq + \text{regret (small)}$$

Claim: The following inequality is true.

$$\max_p p^T M\overline{q} \leq \min_q \overline{p}^T Mq + O(\sqrt{(\log N)/T})$$

Proof.
(Explanation of certain steps included below.)

$$\begin{aligned}
\max_p p^T M\overline{q} &= \max_p \frac{1}{T}\sum_{t=1}^{T} pMq^{(t)} && (1)\\
&\leq \frac{1}{T}\sum_{t=1}^{T} \max_p pMq^{(t)} && (2)\\
&= \frac{1}{T}\sum_{t=1}^{T} p^{(t)}Mq^{(t)} && (3)\\
&\leq \min_q \frac{1}{T}\sum_{t=1}^{T} (p^{(t)})^T Mq + O\left(\sqrt{\frac{\log N}{T}}\right) && (4)\\
&= \min_q \overline{p}^T Mq + O\left(\sqrt{\frac{\log N}{T}}\right) && (5)
\end{aligned}$$

- (1) derives from the definition of $\overline{q}$.
- (2) derives from convexity.
- (3) derives from the fact that the row player, at each time step t, chooses the optimal response, and therefore $p^{(t)}$ is the score-maximizing value for p.
- (4) derives from the multiplicative weights algorithm, via an analysis similar to that of the expected number of mistakes analysis.
- (5) derives from the definition of $\overline{p}$.

Finally, we can conclude the proof of the minimax theorem, knowing this inequality is true.

$$\begin{aligned}
\min_q \max_p p^T Mq &\leq \max_p pM\overline{q} && (1)\\
&\leq \min_q \overline{p}^T Mq + O(\sqrt{(\log N)/T}) && (2)\\
&\leq \max_p \min_q p^T Mq + O(\sqrt{(\log N)/T}) && (3)
\end{aligned}$$

- (1) derives from the definition of the $\min$ function.
- (2) derives directly from the inequality we previously proved.
- (3) derives from the definition of the $\max$ function.

Finally, we note, as before, that $\lim_{T\to\infty} \sqrt{(\log N)/T} = 0$, and therefore

$$\min_q \max_p p^T Mq \leq \max_p \min_q p^T Mq$$

as desired.

□

Approximate Solutions to LPs

I... got too lazy to take notes on this part. Feel free to take a look at these notes from CMU's 15859 offering from Fall 2011, though, if you're interested.

Sources

- Dartmouth CS31
- CMU 15850
- Princeton COS511
- Berkeley ECON 201B